

Animated Pictures for Slide Presentations



Oliver Flatt

University of Washington

Robert Bruce Findler

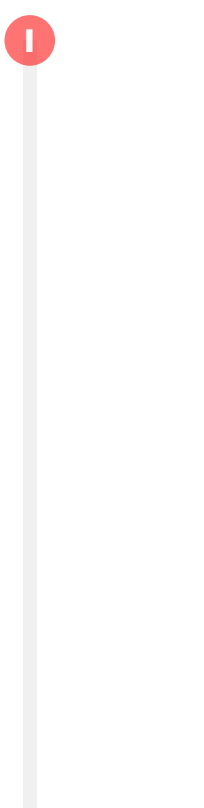
Northwestern University

Matthew Flatt

University of Utah

Outline

Outline



Outline

- I** Motivation
- 2** Making Animations Composable
- 3** Replacing Parts of Animations
- 4** Reflection

What makes a good talk?

What makes a good talk?

- A clear narrative?
- Nice visual examples?

What makes a good talk?

- A clear narrative?
- Nice visual examples?
- Bullet points?

What makes a good talk?

- A clear narrative?
- Nice visual examples?
- Bullet points?
- Lots of explanatory text covering the slide from top to bottom which is meant to be read verbatim by the speaker?

What makes a good talk?

- A clear narrative?
- Nice visual examples?
- Bullet points?
- Lots of explanatory text covering the slide from top to bottom which is meant to be read verbatim by the speaker?
- A series of rhetorical questions?

What makes a good talk?

- A clear narrative?
- Nice visual examples?
- Bullet points?
- Lots of explanatory text covering the slide from top to bottom which is meant to be read verbatim by the speaker?
- A series of rhetorical questions?
- Tasteful font choices?

What makes a good talk?

- A clear narrative?
- Nice visual examples?
- Bullet points?
- Lots of explanatory text covering the slide from top to bottom which is meant to be read verbatim by the speaker?
- A series of rhetorical questions?
- Tasteful font choices?

What makes a good talk?

- A clear narrative? **Nope!**
- Nice visual examples?
- Bullet points?
- Lots of explanatory text covering the slide from top to bottom which is meant to be read verbatim by the speaker?
- A series of rhetorical questions?
- Tasteful font choices?

What makes a good talk is....

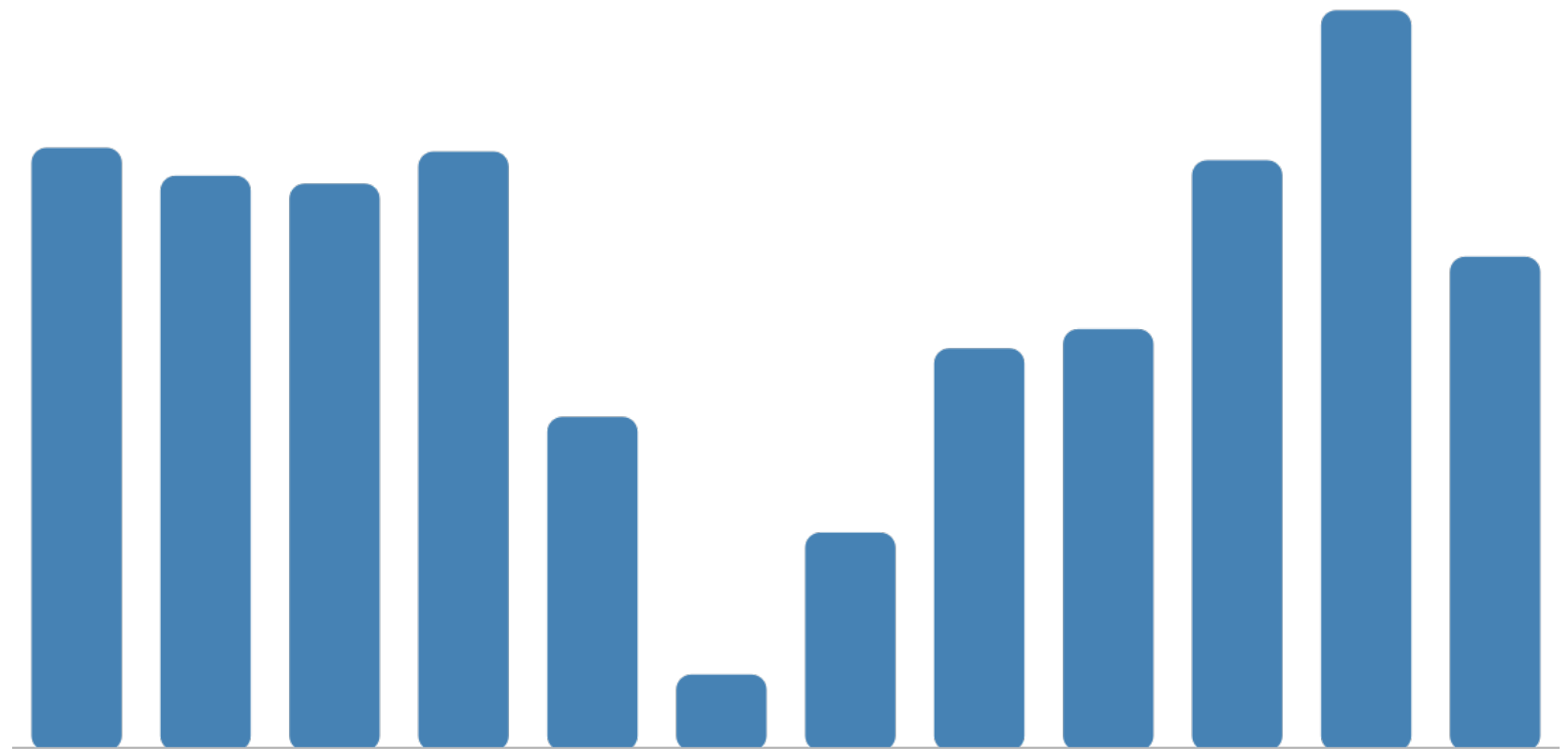
What makes a good talk is....

And if we want our animations to....

|

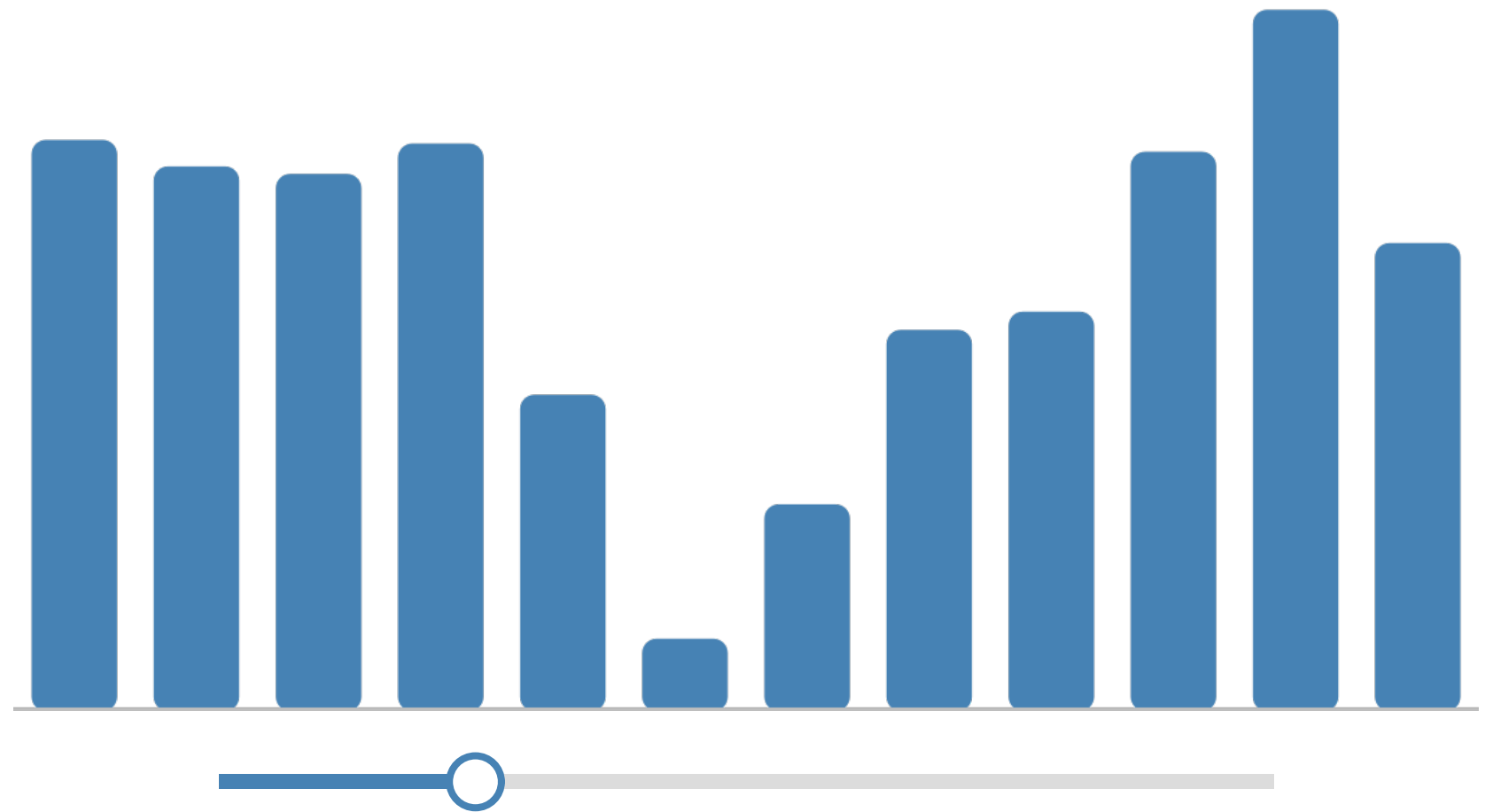
And if we want our animations to....

I) react to real data



And if we want our animations to....

- 1) react to real data
- 2) be interactive

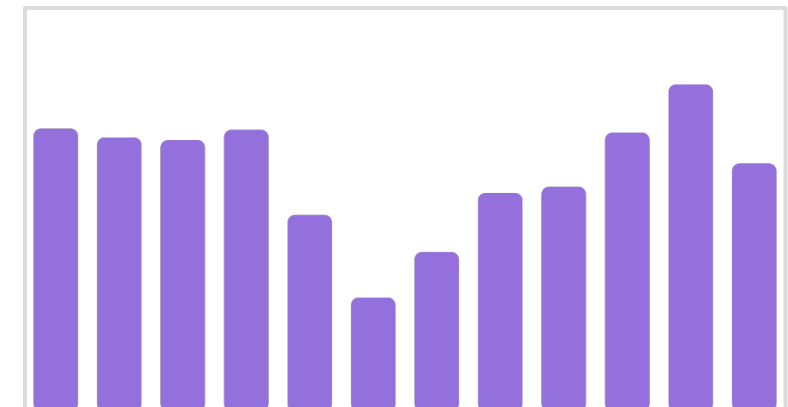
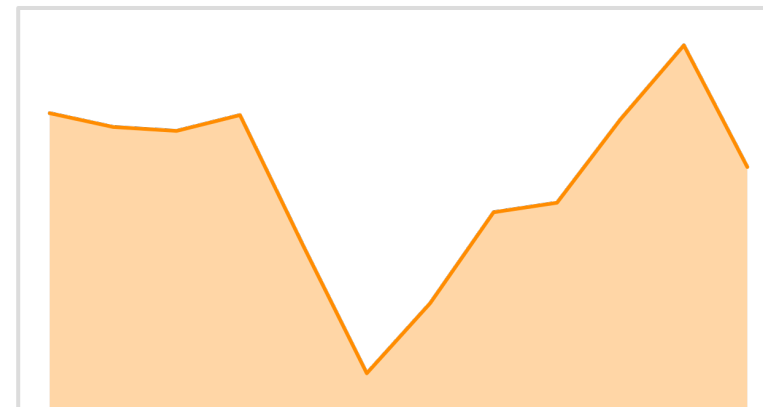
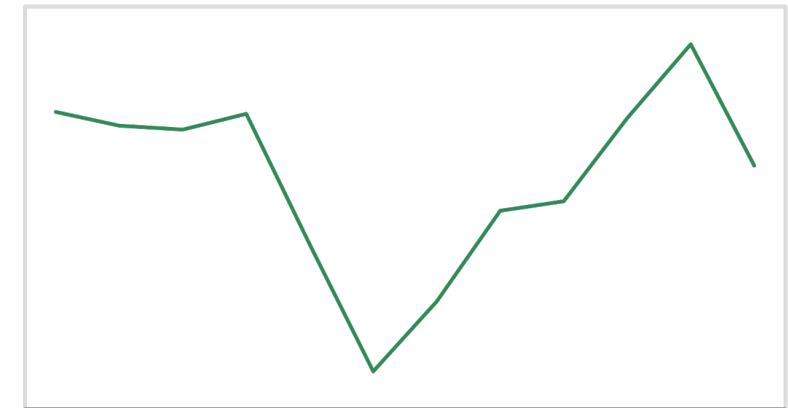
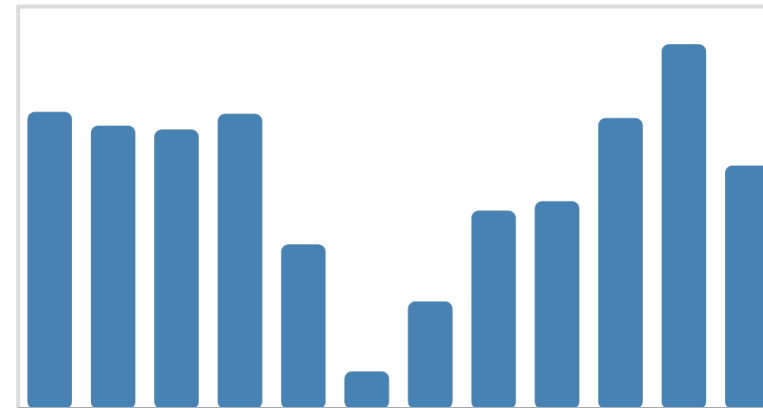


And if we want our animations to....

1) react to real data

2) be interactive

3) be reusable and refactorable

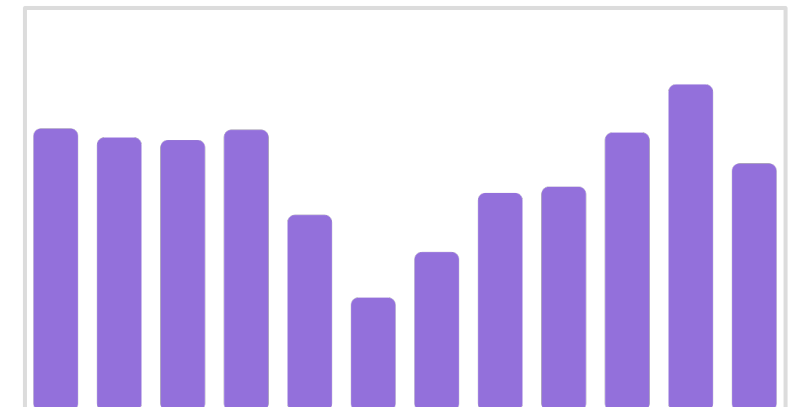
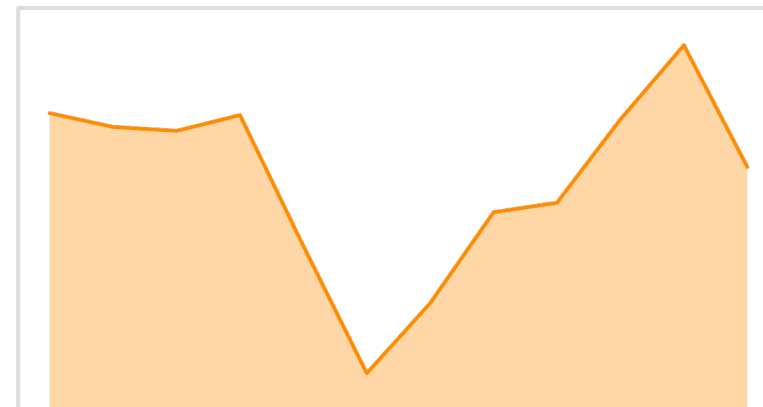
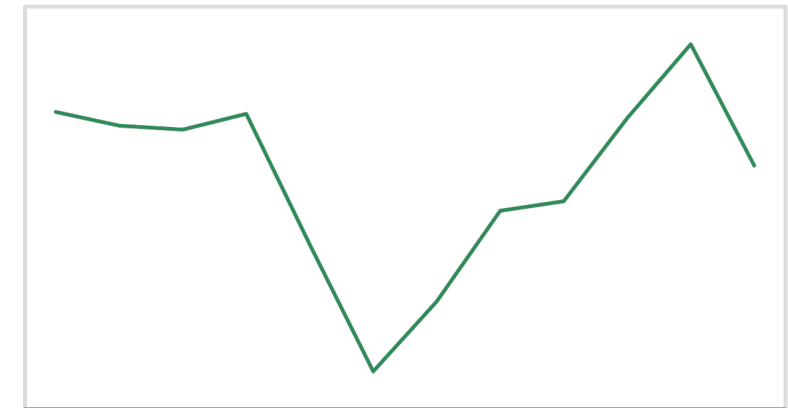
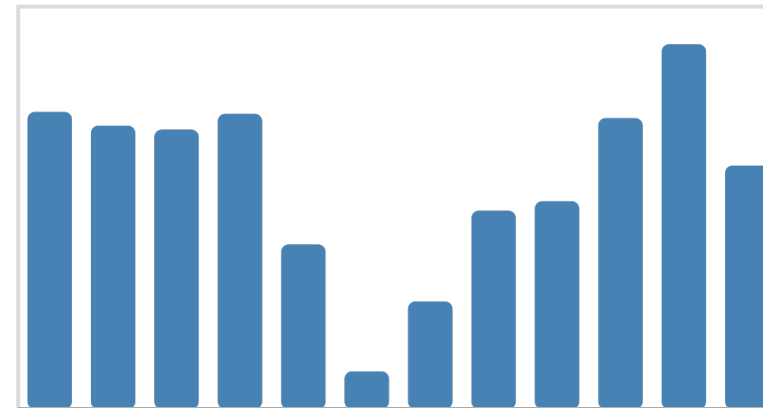


And if we want our animations to....

1) react to real data

2) be interactive

3) be reusable and refactorable



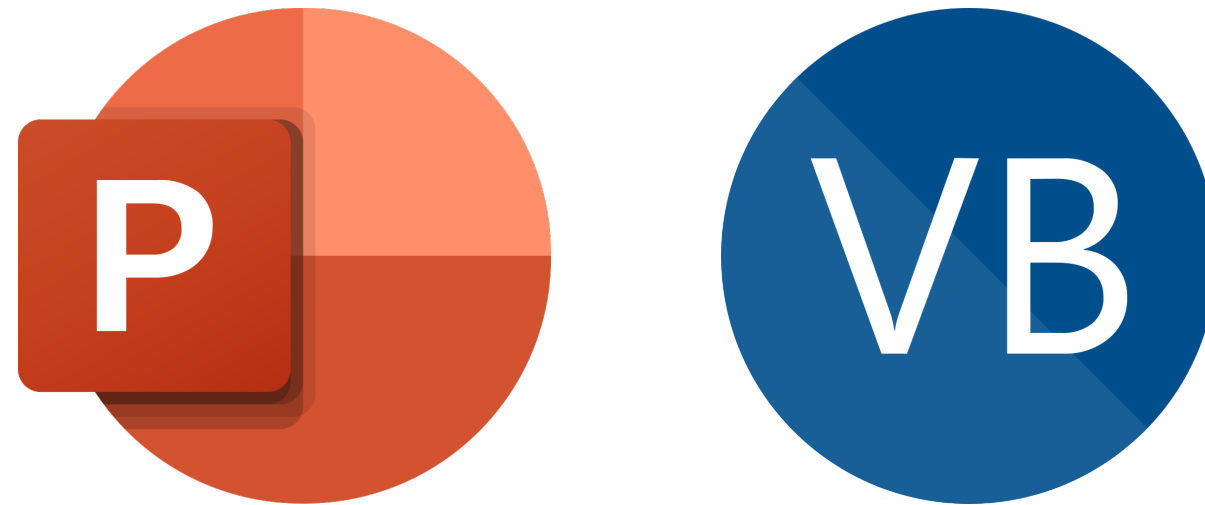
...we have to **CODE**

What to use?

What to use?

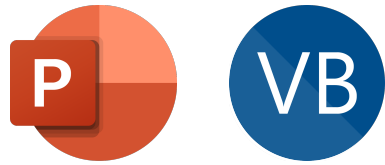


What to use?

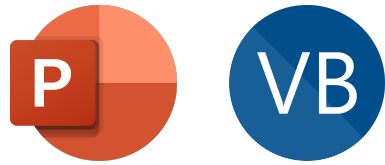


too limited

What to use?



What to use?



L^AT_EX + beamer

What to use?



L^AT_EX + beamer

only overlay steps

What to use?



$\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ + beamer

What to use?



L^AT_EX + beamer



What to use?



L^AT_EX + beamer



arbitrary JavaScript

What to use?



L^AT_EX + beamer



What to use?



L^AT_EX + beamer



Manim



What to use?



L^AT_EX + beamer



Manim

Animation abstraction with limited composability

What to use?



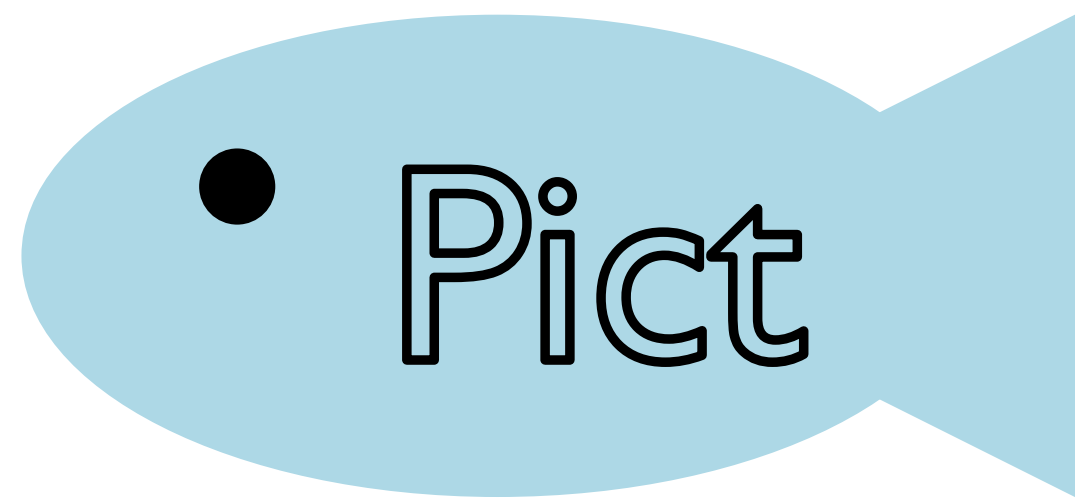
L^AT_EX + beamer



What to use?



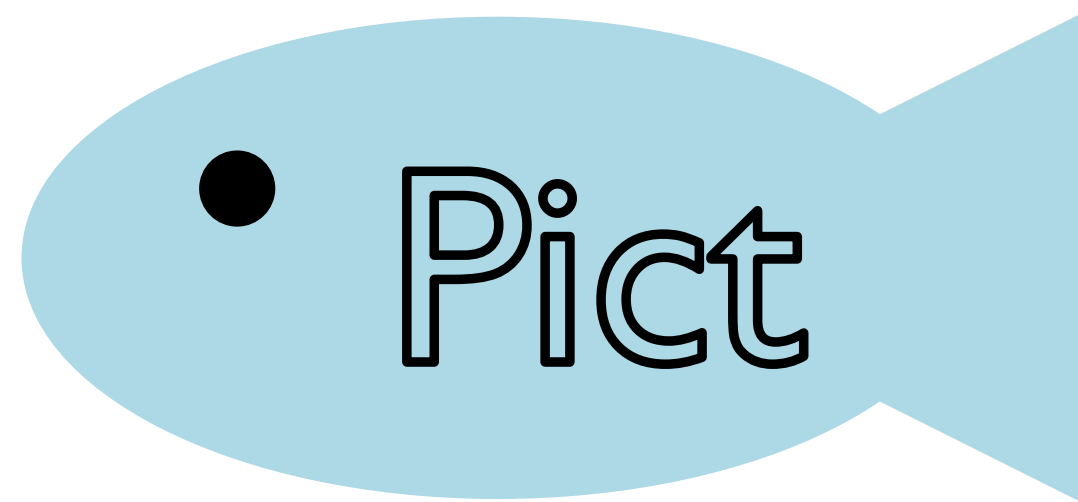
L^AT_EX + beamer



What to use?



L^AT_EX + beamer



fully composable animations

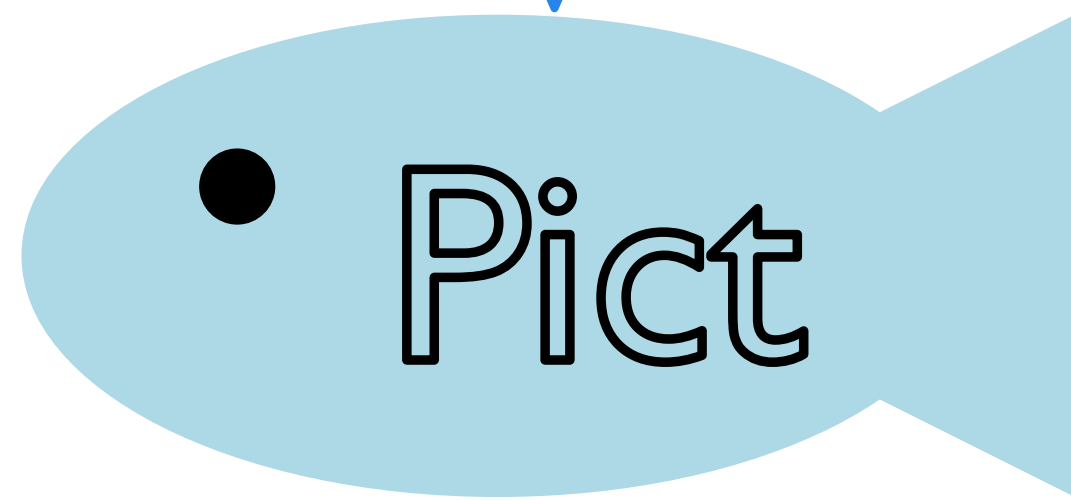
What to use?



L^AT_EX + beamer



this talk!



fully composable animations

Outline

Outline

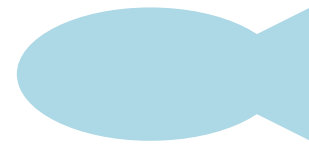
- I** Motivation
- 2 Making Animations Composable
- 3 Replacing Parts of Animations
- 4 Reflection

Outline

- 1 Motivation
- 2 Making Animations Composable
- 3 Replacing Parts of Animations
- 4 Reflection

Static Picts

Static Picts



Static Picts



```
overlay(ellipse(~width: 100, ~height: 50, ~fill: "lightblue"),  
        ~dx: 60, ~dy: 0,  
        triangle(~size: 50, ~fill: "lightblue")  
        .rotate(1/2 * math.pi))
```

this Rhombus expression becomes the fish, one step at a time

Static Picts



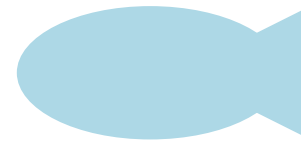
```
overlay(ellipse(~width: 100, ~height: 50, ~fill: "lightblue"),  
        ~dx: 60, ~dy: 0,  
        triangle(~size: 50, ~fill: "lightblue")  
        .rotate(1/2 * math.pi))
```

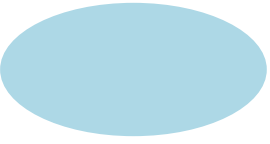
Static Picts



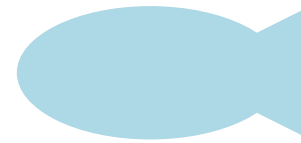
```
overlay(ellipse(~width: 100, ~height: 50, ~fill: "lightblue"),  
        ~dx: 60, ~dy: 0,  
        triangle(~size: 50, ~fill: "lightblue")  
        .rotate(1/2 * math.pi))
```

Static Picts



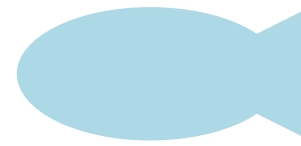
```
overlay(,  
        ~dx: 60, ~dy: 0,  
        triangle(~size: 50, ~fill: "lightblue")  
        .rotate(1/2 * math.pi))
```

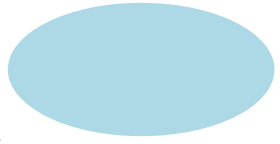
Static Picts



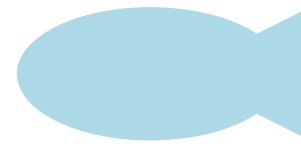
```
overlay(,  
        ~dx: 60, ~dy: 0,  
        triangle(~size: 50, ~fill: "lightblue")  
            .rotate(1/2 * math.pi))
```


Static Picts



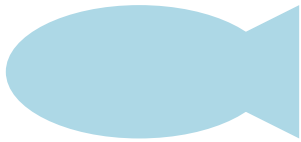
```
overlay(,  
        ~dx: 60, ~dy: 0,  
          
        .rotate(1/2 * math.pi))
```

Static Picts



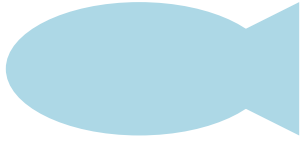
```
overlay(,  
        ~dx: 60, ~dy: 0,  
          
        .rotate(1/2 * math.pi))
```


Static Picts



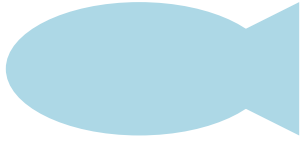
```
overlay(,  
  ~dx: 60, ~dy: 0,  
    
  .rotate(1/2 * 3.141592653589793))
```

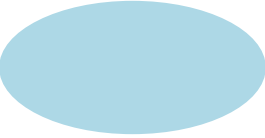
Static Picts



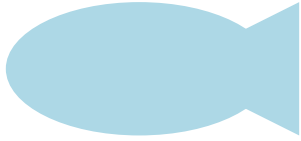
```
overlay(,  
  ~dx: 60, ~dy: 0,  
    
  .rotate(1/2 * 3.141592653589793))
```

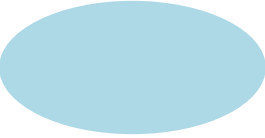
Static Picts



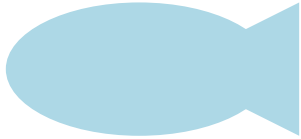
```
overlay(,  
  ~dx: 60, ~dy: 0,  
    
  .rotate(1.5707963267948966))
```

Static Picts



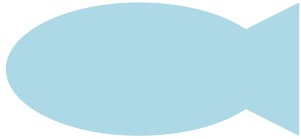
```
overlay(,  
  ~dx: 60, ~dy: 0,  
    
  .rotate(1.5707963267948966))
```

Static Picts



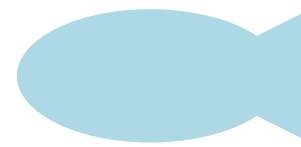
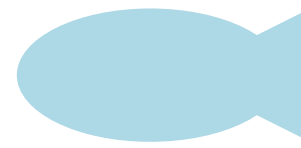
```
overlay(,  
  ~dx: 60, ~dy: 0,  
  )
```

Static Picts



```
overlay(,  
  ~dx: 60, ~dy: 0,  
  )
```


Static Picts



A Generic Tank Function

A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)  
  
beside(tank(red_fish),  
       tank(blue_fish))
```

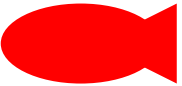
A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)  
  
beside(tank() ,  
       tank(blue_fish))
```

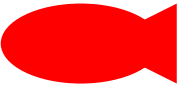
A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)  
  
beside(tank() ,  
       tank(blue_fish))
```

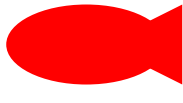
A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
    ~line: "navy", ~fill: water),  
    p)  
  
beside(overlay(rectangle(~width: .width + 20, ~height: .height + 20,  
    ~line: "navy", ~fill: water),  
    ,  
  tank(blue_fish))
```

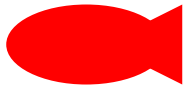
A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
    ~line: "navy", ~fill: water),  
    p)  
  
beside(overlay(rectangle(~width: .width + 20, ~height: .height + 20,  
    ~line: "navy", ~fill: water),  
    ,  
  tank(blue_fish))
```

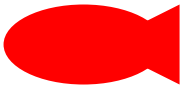
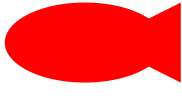

A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)  
  
beside(overlay(rectangle(~width: 66 + 20, ~height: .height + 20,  
                    ~line: "navy", ~fill: water),  
        ,  
      tank(blue_fish))
```

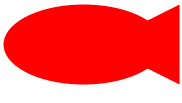
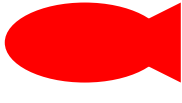
A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)  
  
beside(overlay(rectangle(~width: 66 + 20, ~height: .height + 20,  
                    ~line: "navy", ~fill: water),  
        ,  
      tank(blue_fish))
```


A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)  
  
beside(overlay(rectangle(~width: 86, ~height: .height + 20,  
                    ~line: "navy", ~fill: water),  
        ,  
      tank(blue_fish))
```

A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)  
  
beside(overlay(rectangle(~width: 86, ~height: .height + 20,  
                    ~line: "navy", ~fill: water),  
        ,  
      tank(blue_fish))
```

A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)  
  
beside(overlay(rectangle(~width: 86, ~height: 30 + 20,  
                        ~line: "navy", ~fill: water),  
          ,  
        tank(blue_fish))
```

A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)  
  
beside(overlay(rectangle(~width: 86, ~height: 30 + 20,  
                        ~line: "navy", ~fill: water),  
          ,  
        tank(blue_fish))
```

A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)  
  
beside(overlay(rectangle(~width: 86, ~height: 50,  
                        ~line: "navy", ~fill: water),  
          ,  
        tank(blue_fish))
```

A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)  
  
beside(overlay(rectangle(~width: 86, ~height: 50,  
                        ~line: "navy", ~fill: water),  
          ,  
        tank(blue_fish))
```


A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

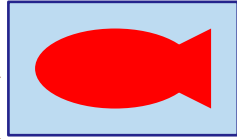
```
beside(overlay(,  
           ,  
        tank(blue_fish)))
```

A Generic Tank Function

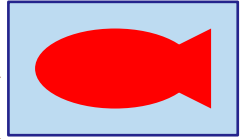
```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

```
beside(overlay(,  
            ,  
      tank(blue_fish)))
```


A Generic Tank Function

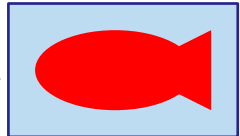
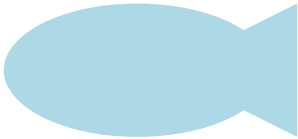
```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)  
  
beside(,  
      tank(blue_fish))
```

A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)  
  
beside(,  
      tank(blue_fish))
```

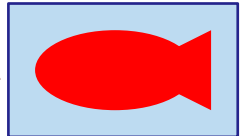
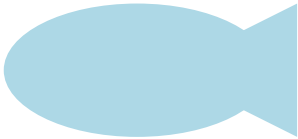
A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

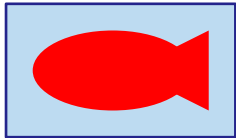
```
beside(,  
      tank())
```

A Generic Tank Function

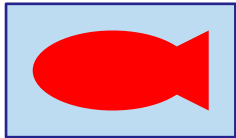
```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

```
beside(,  
      tank())
```

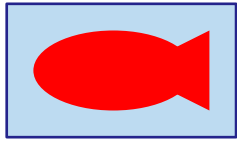
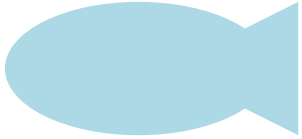
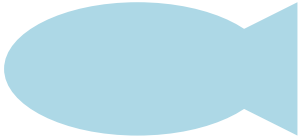
A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
    ~line: "navy", ~fill: water),  
    p)  
  
beside(,  
  overlay(rectangle(~width: .width + 20, ~height: .height + 20,  
    ~line: "navy", ~fill: water),  
    ))
```

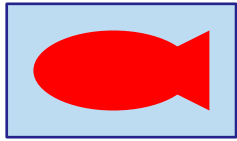
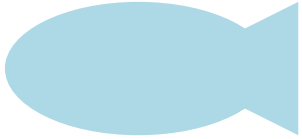
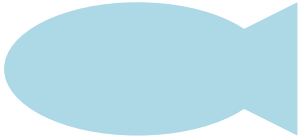
A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
    ~line: "navy", ~fill: water),  
    p)  
  
beside(,  
  overlay(rectangle(~width: .width + 20, ~height: .height + 20,  
    ~line: "navy", ~fill: water),  
      
  ))
```


A Generic Tank Function

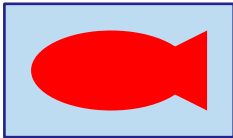
```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
    ~line: "navy", ~fill: water),  
    p)  
  
beside(,  
  overlay(rectangle(~width: 110 + 20, ~height: .height + 20,  
    ~line: "navy", ~fill: water),  
    ))
```

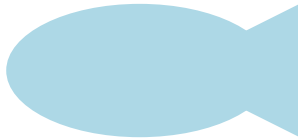
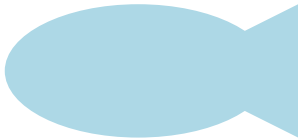
A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
    ~line: "navy", ~fill: water),  
    p)  
  
beside(,  
  overlay(rectangle(~width: 110 + 20, ~height: .height + 20,  
    ~line: "navy", ~fill: water),  
    ))
```


A Generic Tank Function

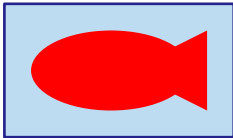
```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

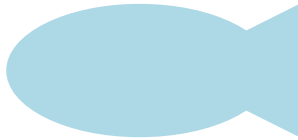
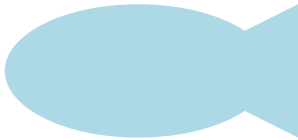
beside(,

```
  overlay(rectangle(~width: 130, ~height: .height + 20,  
          ~line: "navy", ~fill: water),  
  ))
```

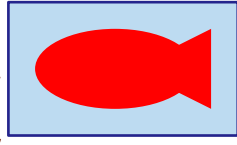
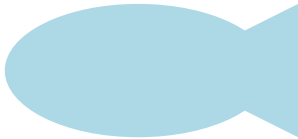
A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

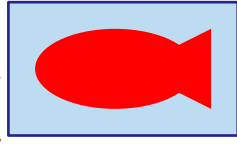
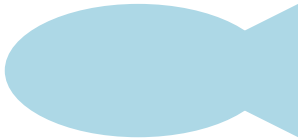
beside(,

```
  overlay(rectangle(~width: 130, ~height: .height + 20,  
          ~line: "navy", ~fill: water),  
  ))
```

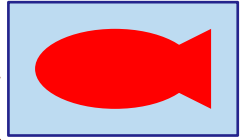
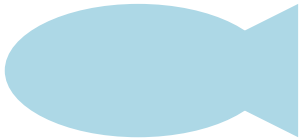
A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
    ~line: "navy", ~fill: water),  
    p)  
  
beside(,  
  overlay(rectangle(~width: 130, ~height: 50 + 20,  
    ~line: "navy", ~fill: water),  
      
  ))
```

A Generic Tank Function

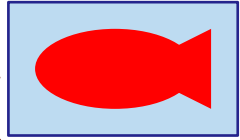
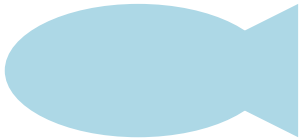
```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
    ~line: "navy", ~fill: water),  
    p)  
  
beside(,  
  overlay(rectangle(~width: 130, ~height: 50 + 20,  
    ~line: "navy", ~fill: water),  
      
  ))
```

A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
    ~line: "navy", ~fill: water),  
    p)  
  
beside(,  
  overlay(rectangle(~width: 130, ~height: 70,  
    ~line: "navy", ~fill: water),  
      
  ))
```

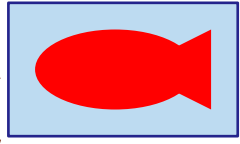
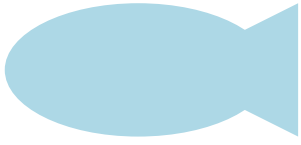
A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

```
beside(,  
      overlay(rectangle(~width: 130, ~height: 70,  
                        ~line: "navy", ~fill: water),  
                
            ))
```

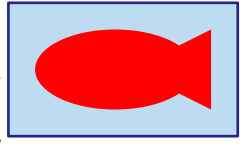
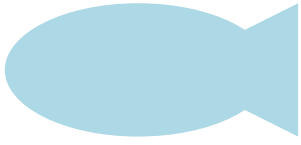

A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

```
beside(,  
      overlay(,  
              )
```

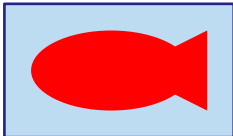
A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

```
beside(,  
      overlay(,  
              )
```

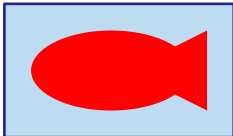

A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

```
beside(,  
      )
```

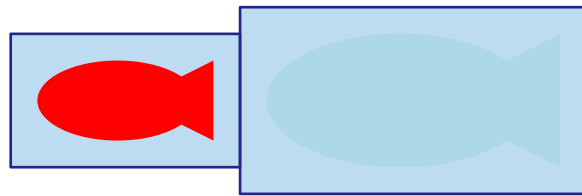
A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

```
beside(,  
      )
```

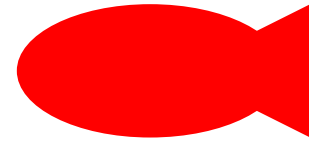
A Generic Tank Function

```
fun tank(p):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

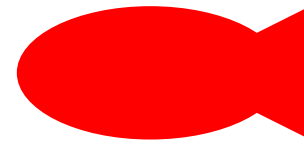


Animations as Functions of Time

Animations as Functions of Time

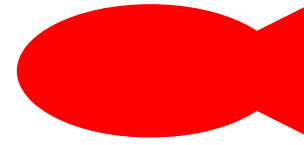


Animations as Functions of Time



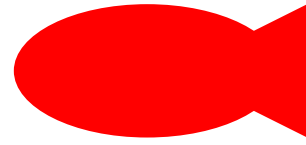
play an animation, then play it in reverse

Animations as Functions of Time



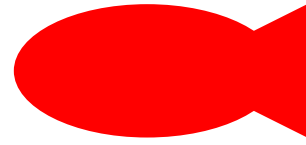
play an animation, then play it in reverse

Animations as Functions of Time



```
fun spinny_fish_func(t):  
    fish("red", 50).rotate(t * 2 * math.pi)
```

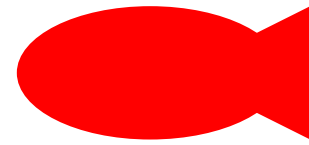

Animations as Functions of Time



```
fun spinny_fish_func(t):  
    fish("red", 50).rotate(t * 2 * math.pi)
```

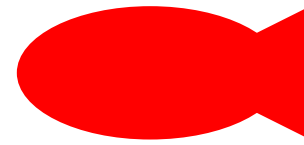
function over time t in $[0, 1]$

Animations as Functions of Time

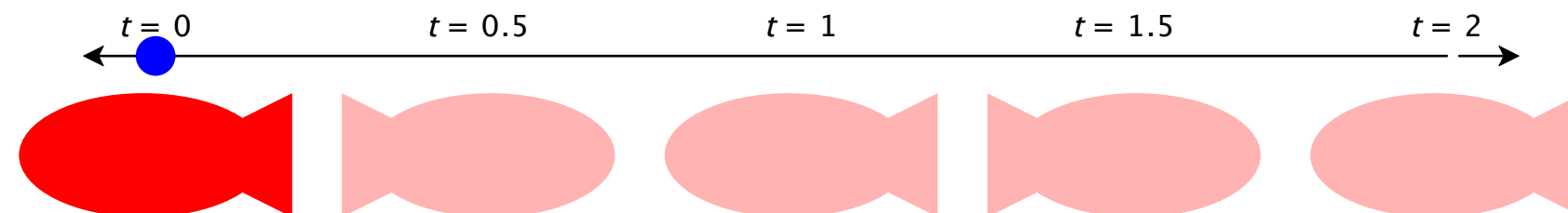


```
fun spinny_fish_func(t):  
    fish("red", 50).rotate(t * 2 * math.pi)  
  
fun spin_back(t):  
    if t < 1  
    | spinny_fish_func(t)  
    | spinny_fish_func(2 - t)
```

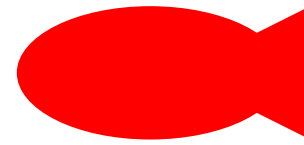
Animations as Functions of Time



```
fun spinny_fish_func(t):  
    fish("red", 50).rotate(t * 2 * math.pi)  
  
fun spin_back(t):  
    if t < 1  
    | spinny_fish_func(t)  
    | spinny_fish_func(2 - t)
```

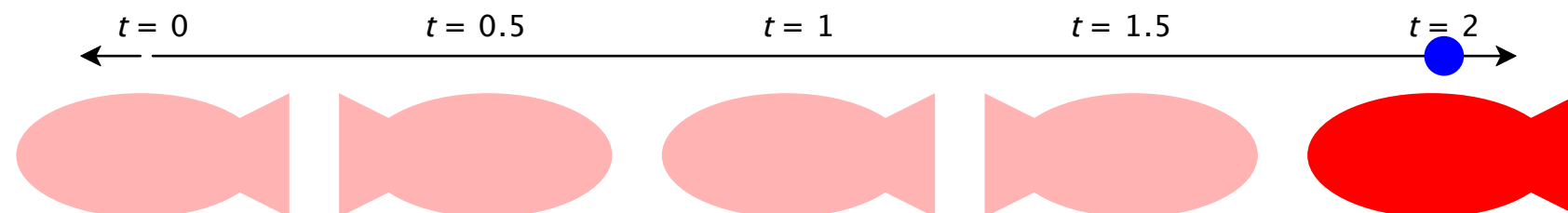


Animations as Functions of Time

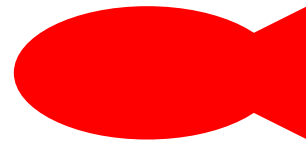


```
fun spinny_fish_func(t):  
    fish("red", 50).rotate(t * 2 * math.pi)
```

```
fun spin_back(t):  
    if t < 1  
    | spinny_fish_func(t)  
    | spinny_fish_func(2 - t)
```



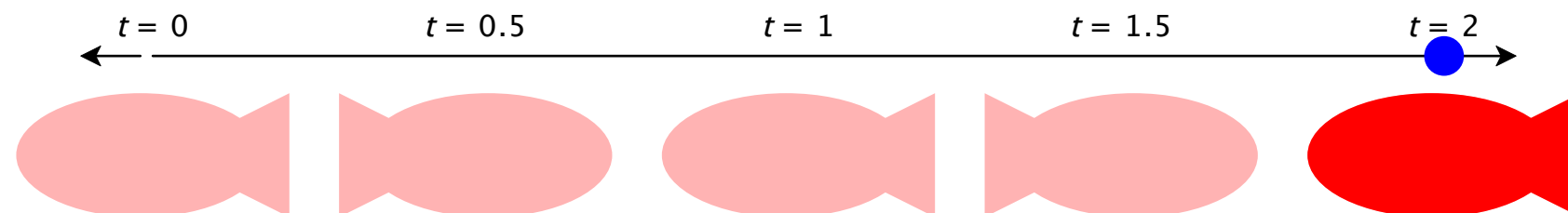
Animations as Functions of Time



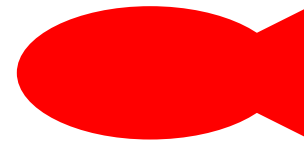
```
fun spinny_fish_func(t):  
    fish("red", 50).rotate(t * 2 * math.pi)
```

```
fun spin_back(t):  
    if t < 1  
    | spinny_fish_func(t)  
    | spinny_fish_func(2 - t)
```

how to generalize?

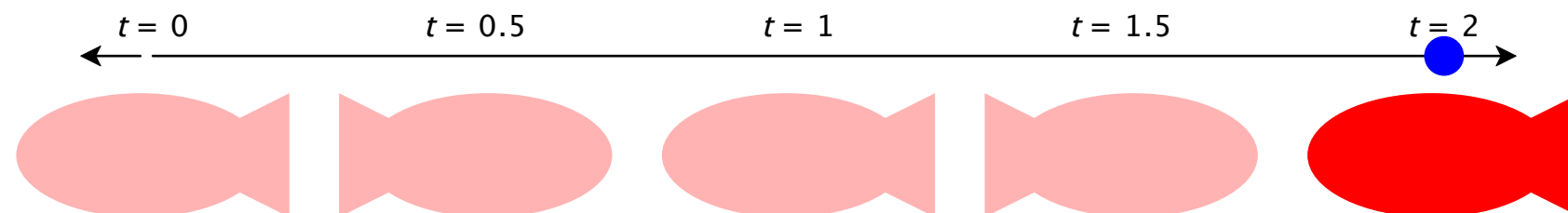


Animations as Functions of Time

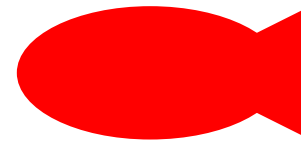


```
fun spinny_fish_func(t):  
    fish("red", 50).rotate(t * 2 * math.pi)
```

```
fun spin_back(f, t):  
    if t < 1  
    | f(t)  
    | f(2 - t)
```

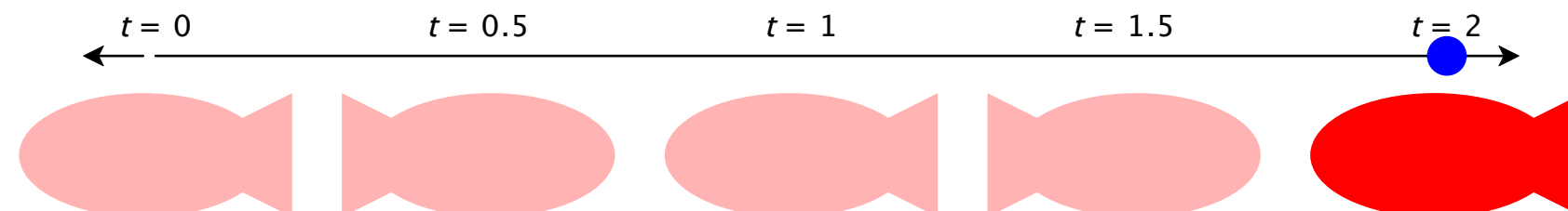


Animations as Functions of Time

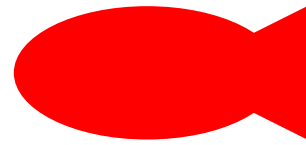


```
fun spinny_fish_func(t):  
    fish("red", 50).rotate(t * 2 * math.pi)
```

```
fun spin_back(f, t):  
    if t < ???  
    | f(t)  
    | f(??? - t)
```



Animations as Functions of Time

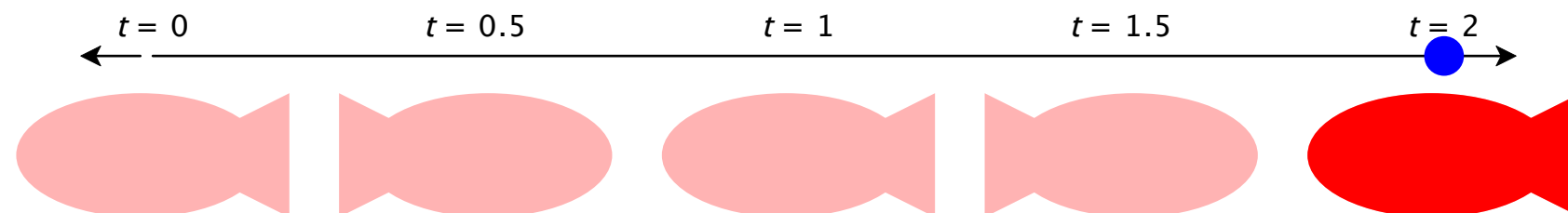


```
fun spinny_fish_func(t):  
    fish("red", 50).rotate(t * 2 * math.pi)
```

```
fun spin_back(f, t):
```

```
    if t < ???  
    | f(t)  
    | f(??? - t)
```

need duration of f!



Built-in Animation Abstraction

Built-in Animation Abstraction

```
def spinny_fish:  
    animate(  
        ~extent: 2,  
        fun (t):  
            fish("red", 50).rotate(t * 2 * math.pi)  
    )
```

Built-in Animation Abstraction

time info built in to Pict DSL

```
def spinny_fish:  
  animate(  
    ~extent: 2,  
    fun (t):  
      fish("red", 50).rotate(t * 2 * math.pi)  
  )
```

Built-in Animation Abstraction

```
def spinny_fish:  
    animate(  
        ~extent: 2, lasts 2 seconds  
        fun (t):  
            fish("red", 50).rotate(t * 2 * math.pi)  
    )
```

Built-in Animation Abstraction

```
def spinny_fish:
  animate(
    ~extent: 2,
    fun (t):
      fish("red", 50).rotate(t * 2 * math.pi)
  )

fun play_then_reverse(p):
  switch(p, reverse(p))
```

Built-in Animation Abstraction

```
def spinny_fish:
  animate(
    ~extent: 2,
    fun (t):
      fish("red", 50).rotate(t * 2 * math.pi)
  )

fun play_then_reverse(p):
  switch(p, reverse(p))

play_then_reverse(spinny_fish)
```


Built-in Animation Abstraction

```
def spinny_fish:
  animate(
    ~extent: 2,
    fun (t):
      fish("red", 50).rotate(t * 2 * math.pi)
  )

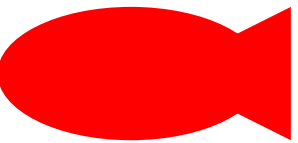
fun play_then_reverse(p):
  switch(p, reverse(p))

play_then_reverse(spinny_fish)
```

Built-in Animation Abstraction

```
def spinny_fish:  
    animate(  
        ~extent: 2,  
        fun (t):  
            fish("red", 50).rotate(t * 2 * math.pi)  
    )
```

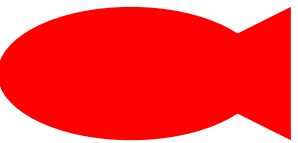
```
fun play_then_reverse(p):  
    switch(p, reverse(p))
```

```
play_then_reverse()
```


Built-in Animation Abstraction

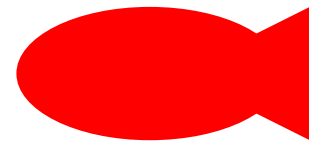
```
def spinny_fish:  
    animate(  
        ~extent: 2,  
        fun (t):  
            fish("red", 50).rotate(t * 2 * math.pi)  
    )
```

```
fun play_then_reverse(p):  
    switch(p, reverse(p))
```

```
play_then_reverse()
```

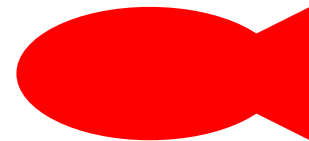
Built-in Animation Abstraction

```
def spinny_fish:  
    animate(  
        ~extent: 2,  
        fun (t):  
            fish("red", 50).rotate(t * 2 * math.pi)  
    )  
  
fun play_then_reverse(p):  
    switch(p, reverse(p))
```



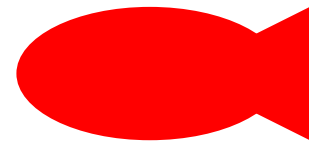
Built-in Animation Abstraction

```
def spinny_fish:  
    animate(  
        ~extent: 2,  
        fun (t):  
            fish("red", 50).rotate(t * 2 * math.pi)  
    )  
  
fun play_then_reverse(p):  
    switch(p, reverse(p))
```

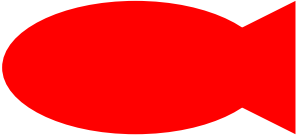


Built-in Animation Abstraction

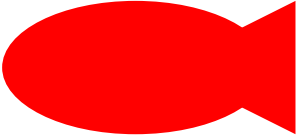
```
def spinny_fish:  
    animate(  
        ~extent: 2,  
        fun (t):  
            fish("red", 50).rotate(t * 2 * math.pi)  
        )  
  
fun play_then_reverse(p):  
    switch(p, reverse(p))
```



Built-in Animation Abstraction

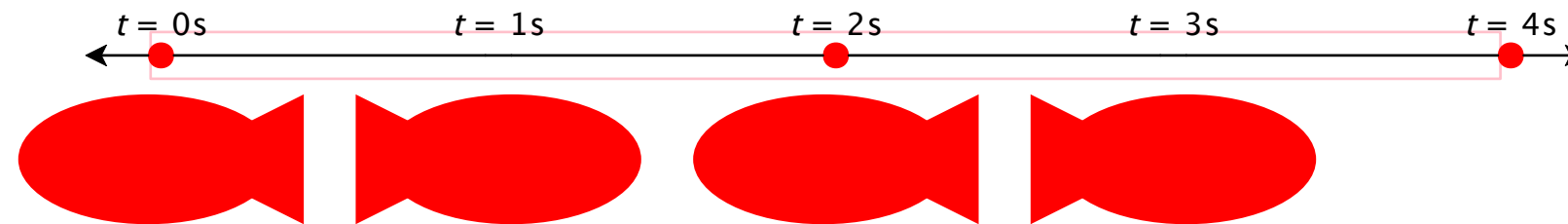
```
def spinny_fish:  
    animate(  
        ~extent: 2,  
        fun (t):  
            fish("red", 50).rotate(t * 2 * math.pi)  
    )  
  
fun play_then_reverse(p):  
    switch(p, reverse(p))  
  
explain_anim()
```

Built-in Animation Abstraction

```
def spinny_fish:  
    animate(  
        ~extent: 2,  
        fun (t):  
            fish("red", 50).rotate(t * 2 * math.pi)  
    )  
  
fun play_then_reverse(p):  
    switch(p, reverse(p))  
  
explain_anim()
```


Built-in Animation Abstraction

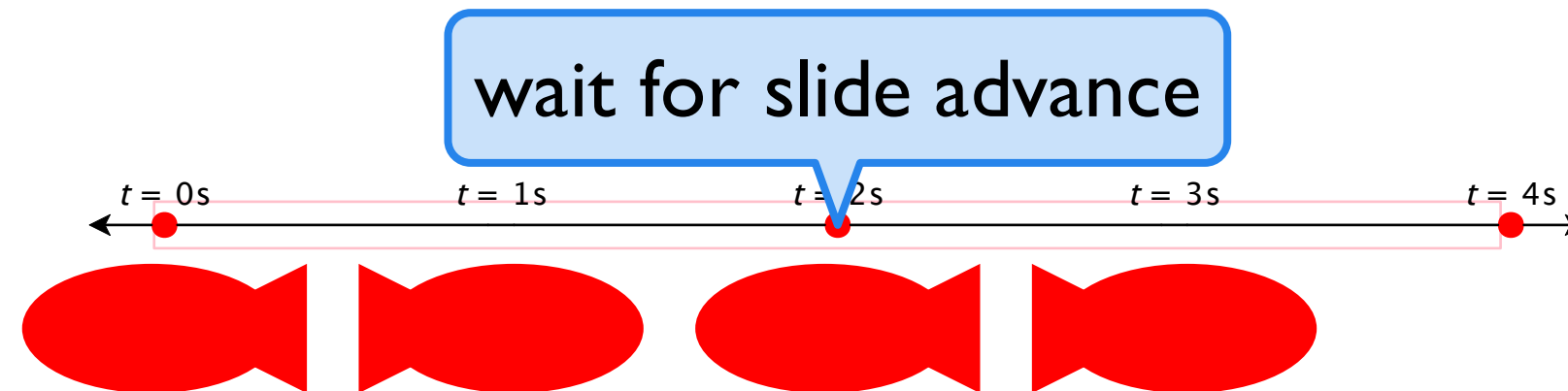
```
def spinny_fish:  
    animate(  
        ~extent: 2,  
        fun (t):  
            fish("red", 50).rotate(t * 2 * math.pi)  
        )  
  
fun play_then_reverse(p):  
    switch(p, reverse(p))
```



Built-in Animation Abstraction

```
def spinny_fish:  
    animate(  
        ~extent: 2,  
        fun (t):  
            fish("red", 50).rotate(t * 2 * math.pi)  
    )
```

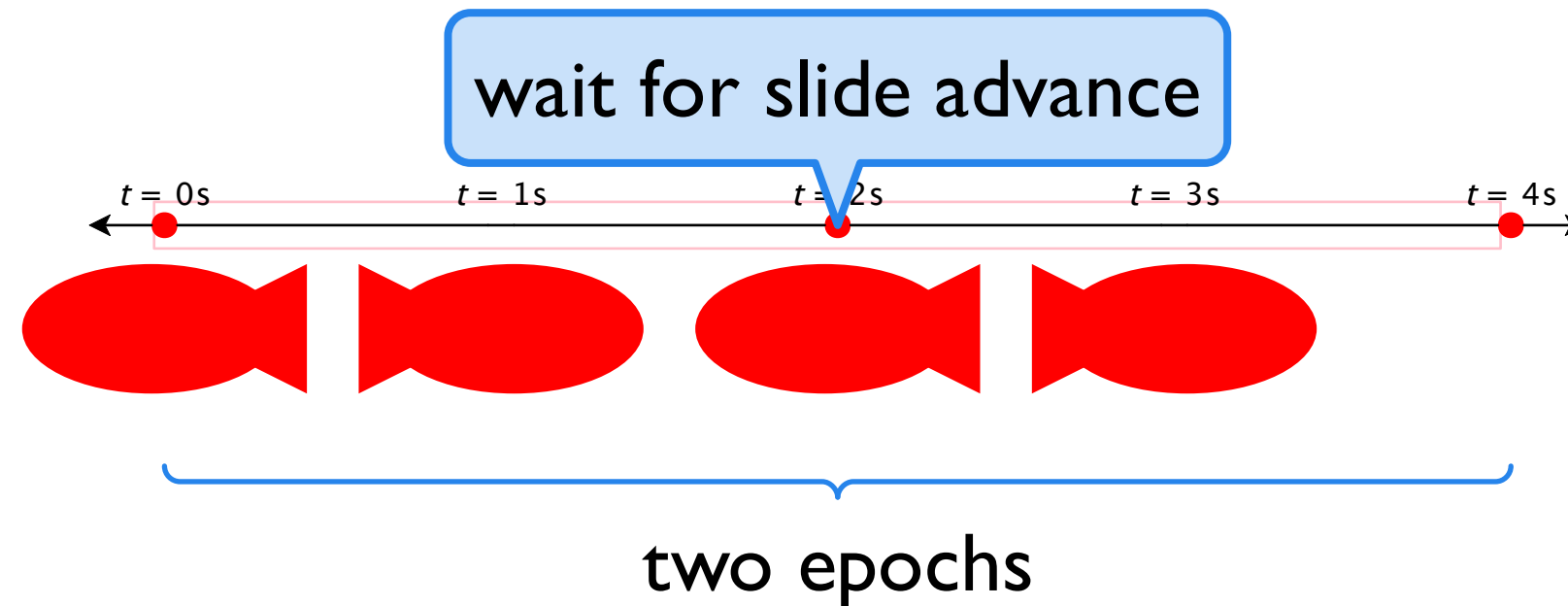
```
fun play_then_reverse(p):  
    switch(p, reverse(p))
```



Built-in Animation Abstraction

```
def spinny_fish:  
    animate(  
        ~extent: 2,  
        fun (t):  
            fish("red", 50).rotate(t * 2 * math.pi)  
    )
```

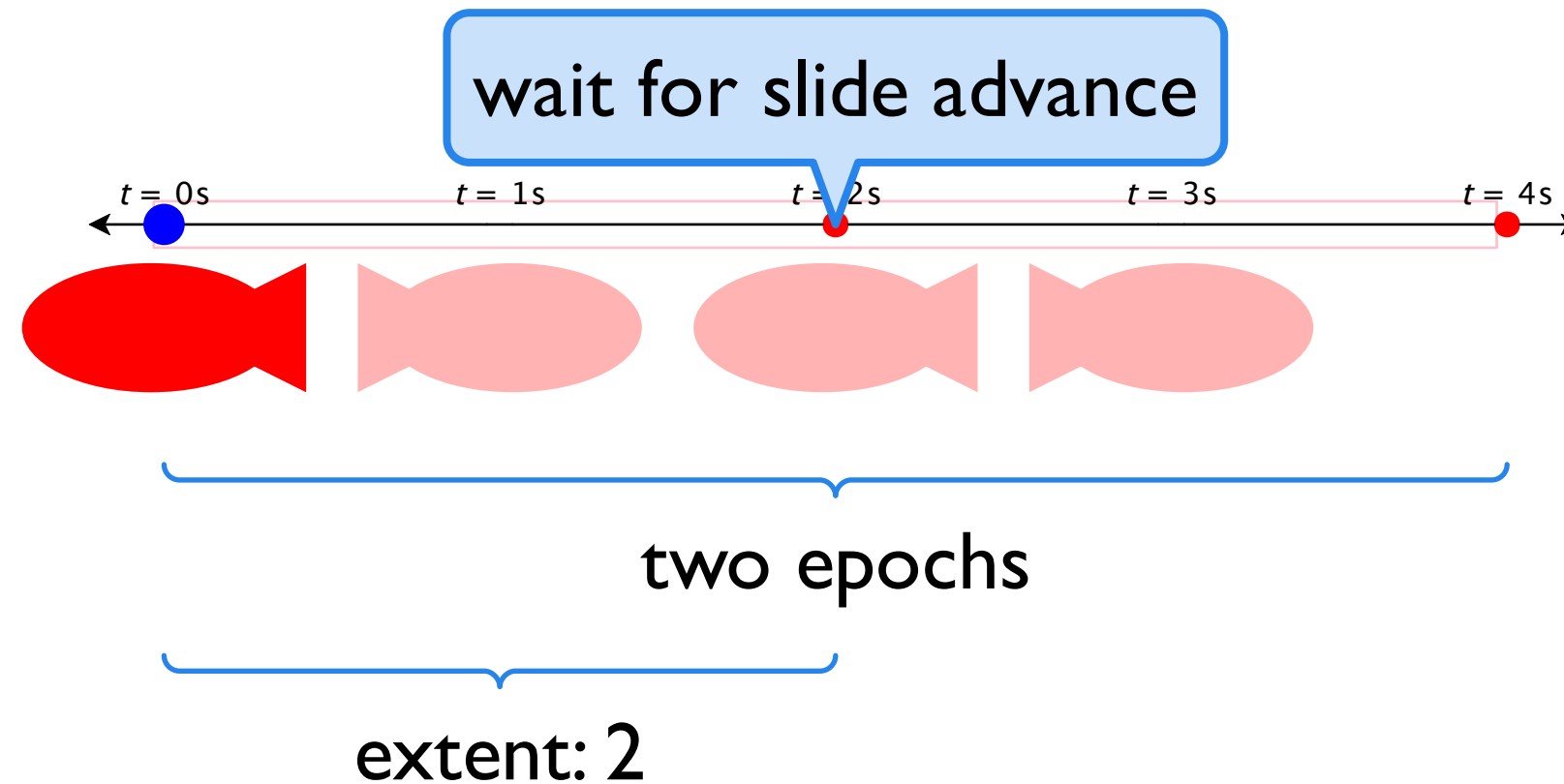
```
fun play_then_reverse(p):  
    switch(p, reverse(p))
```



Built-in Animation Abstraction

```
def spinny_fish:  
    animate(  
        ~extent: 2,  
        fun (t):  
            fish("red", 50).rotate(t * 2 * math.pi)  
    )
```

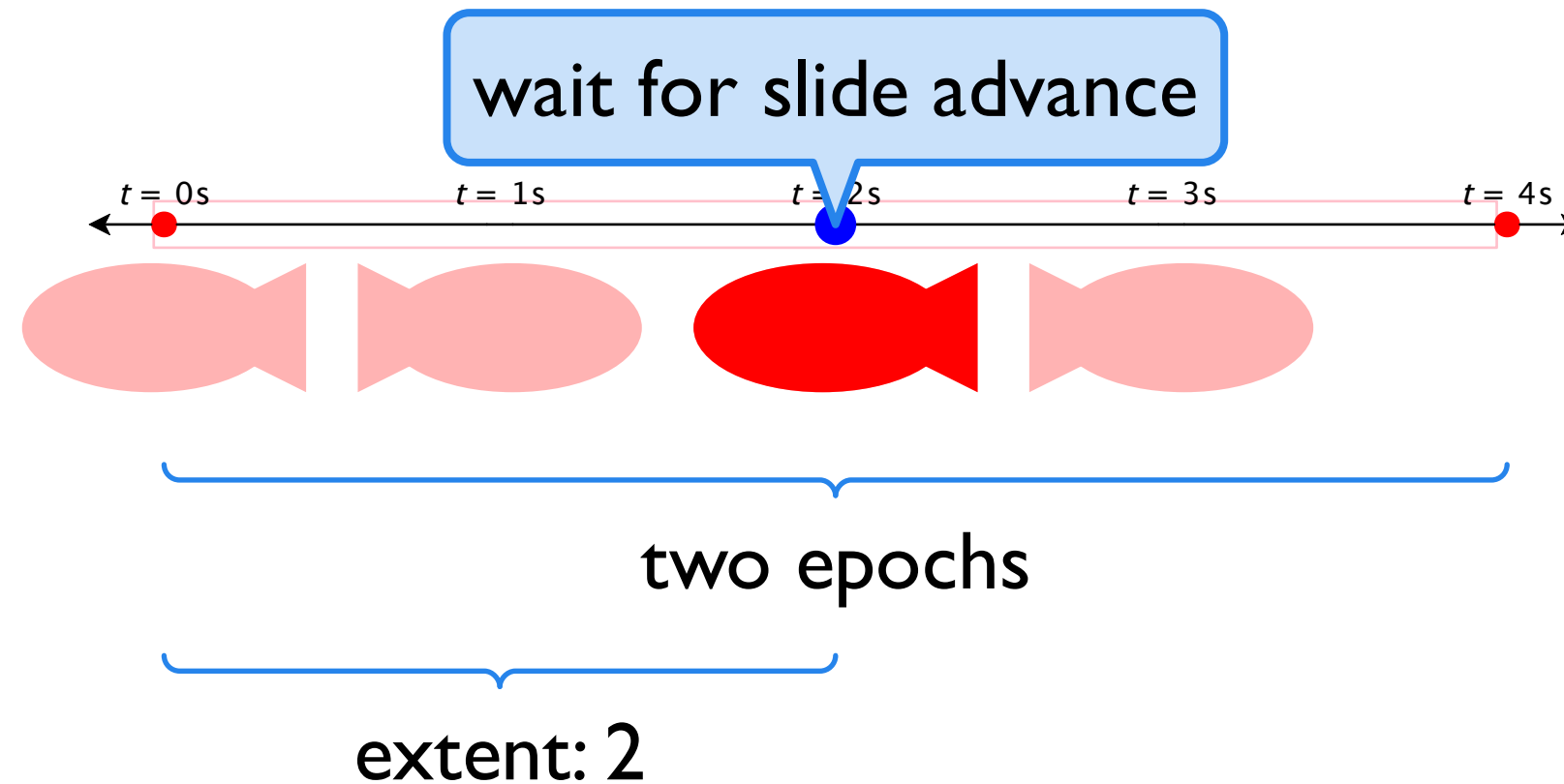
```
fun play_then_reverse(p):  
    switch(p, reverse(p))
```



Built-in Animation Abstraction

```
def spinny_fish:  
    animate(  
        ~extent: 2,  
        fun (t):  
            fish("red", 50).rotate(t * 2 * math.pi)  
    )
```

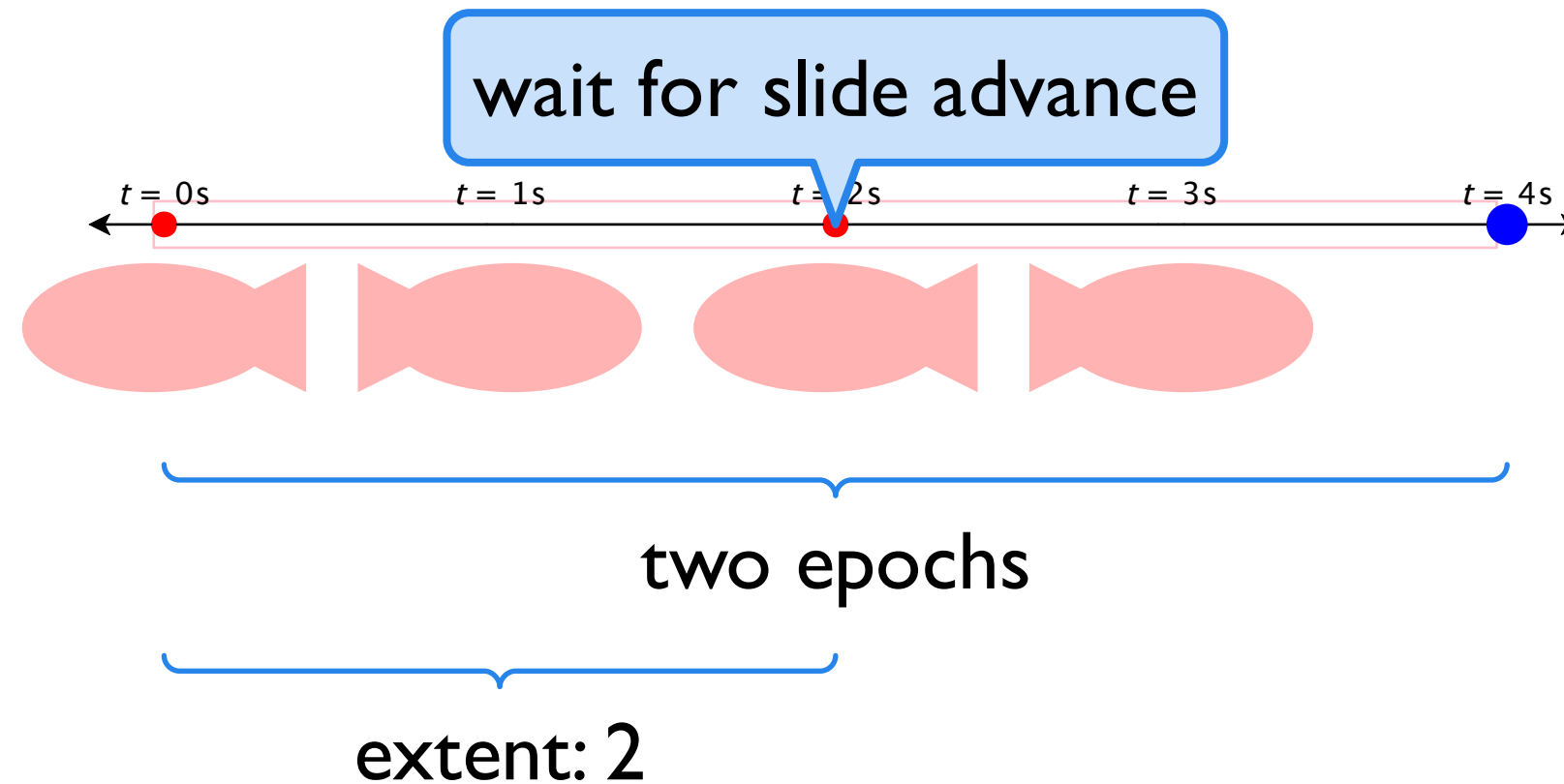
```
fun play_then_reverse(p):  
    switch(p, reverse(p))
```



Built-in Animation Abstraction

```
def spinny_fish:  
    animate(  
        ~extent: 2,  
        fun (t):  
            fish("red", 50).rotate(t * 2 * math.pi)  
    )
```

```
fun play_then_reverse(p):  
    switch(p, reverse(p))
```



All Picts are Animated

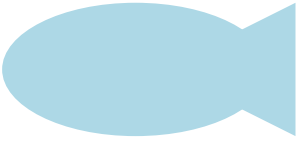
All Picts are Animated

```
stack(blue_fish, blue_fish)
```

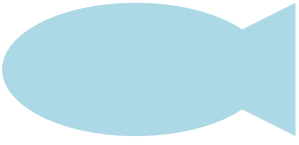

All Picts are Animated

```
stack(blue_fish, blue_fish)
```

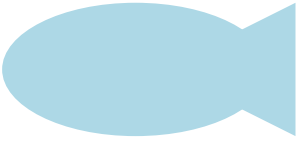
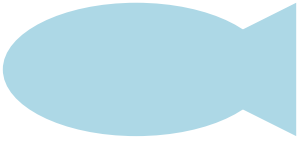
All Picts are Animated

```
stack(, blue_fish)
```

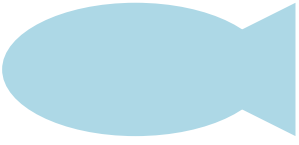
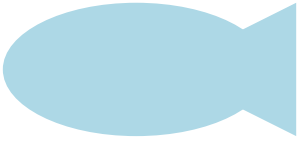

All Picts are Animated

```
stack(, blue_fish)
```

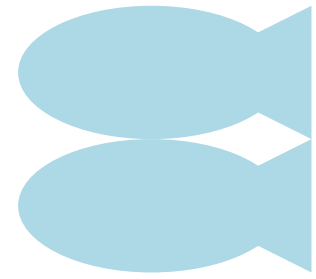
All Picts are Animated

stack(, )

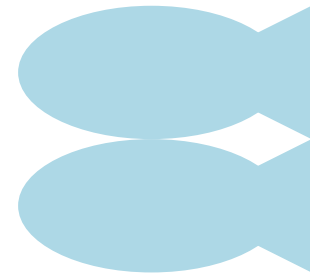
All Picts are Animated

stack(, )

All Picts are Animated

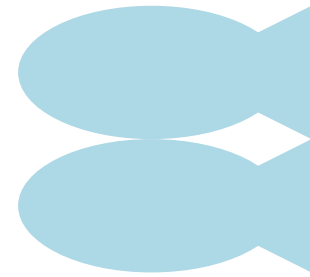


All Picts are Animated



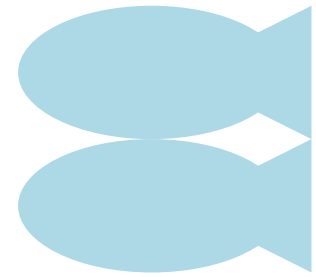
```
stack(spinny_fish, spinny_fish)
```

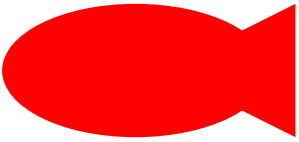
All Picts are Animated



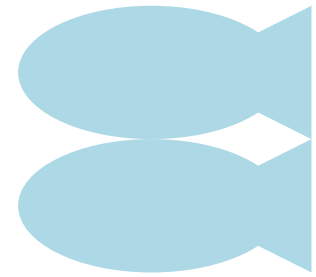
```
stack(spinny_fish, spinny_fish)
```

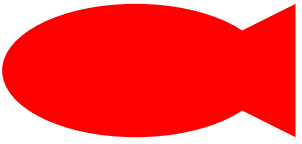

All Picts are Animated



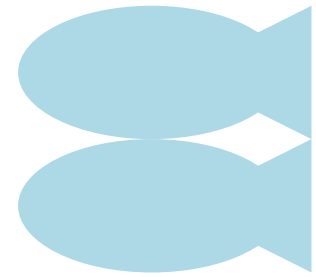
stack(, spinny_fish)

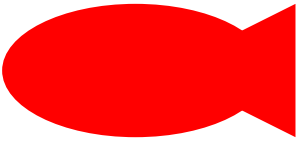
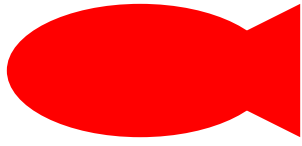
All Picts are Animated



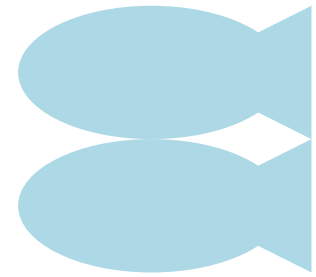
stack(, spinny_fish)

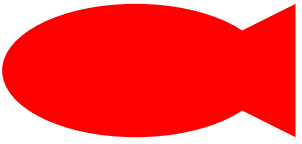
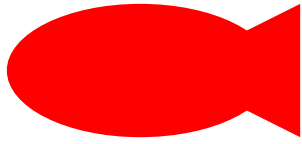
All Picts are Animated



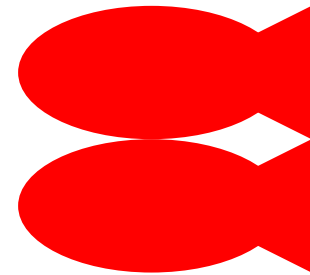
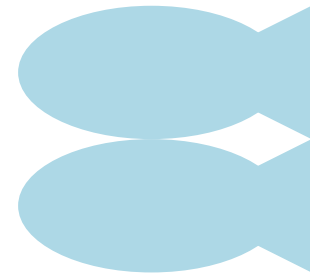
stack(, )

All Picts are Animated

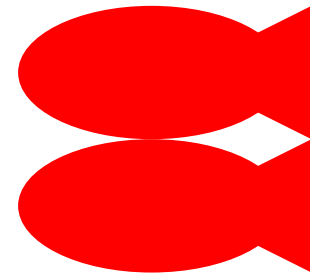
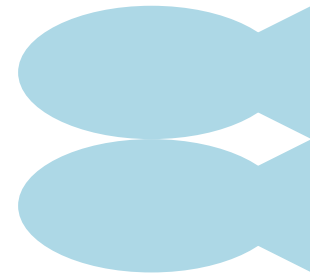


stack(, )

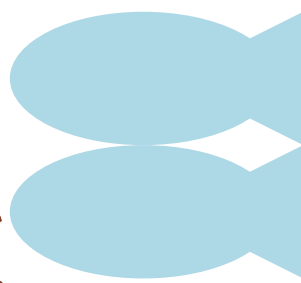
All Picts are Animated

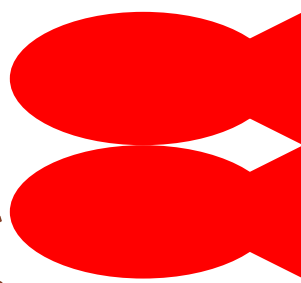


All Picts are Animated

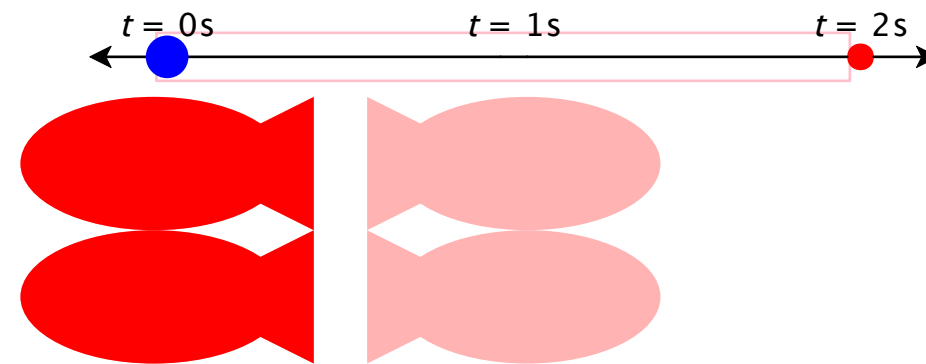
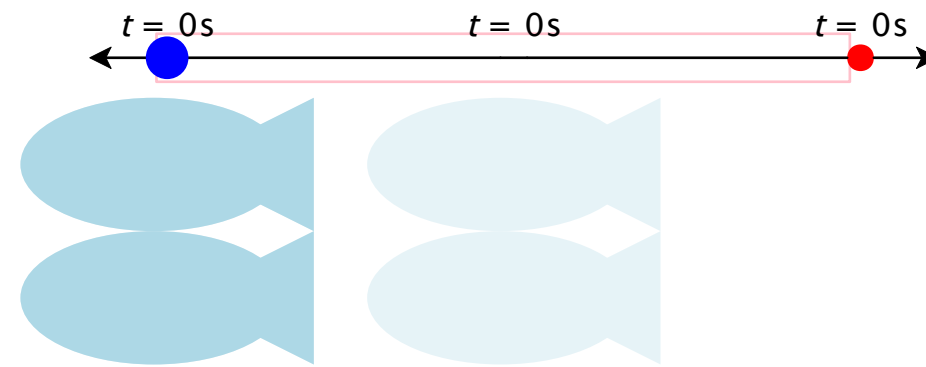


All Picts are Animated

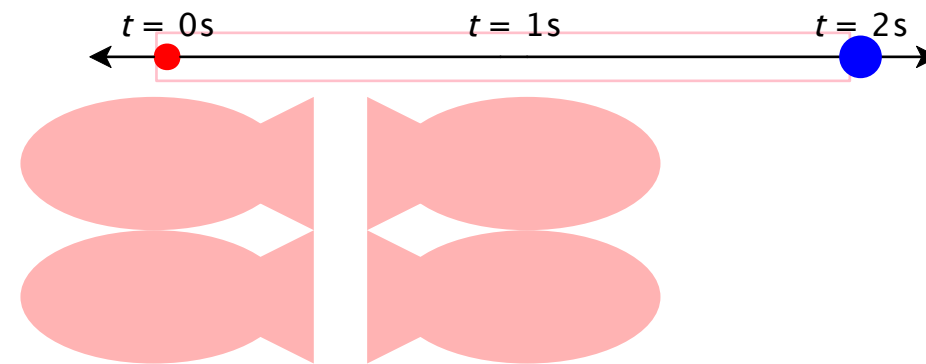
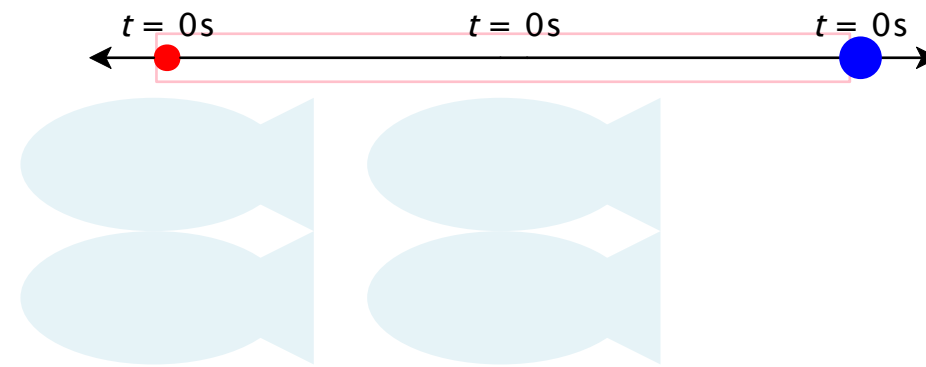
explain_anim()

explain_anim()

All Picts are Animated



All Picts are Animated



Outline

Outline

- 1 Motivation
- 2 Making Animations Composable
- 3 Replacing Parts of Animations
- 4 Reflection

Outline

- 1 Motivation
- 2 Making Animations Composable
- 3 Replacing Parts of Animations**
- 4 Reflection

Replacing Part of a Pict

Replacing Part of a Pict

```
def spike_ring:
  overlay(& (for List (i in 0..spikes):
    def angle = 2 * math.pi * i / spikes
    line(~dx: spike_len * math.cos(angle),
      ~dy: spike_len * math.sin(angle),
      ~line: "purple", ~line_width: 4)))

def urchin = overlay(spike_ring, circle(~size: 30, ~fill: "purple"))
```

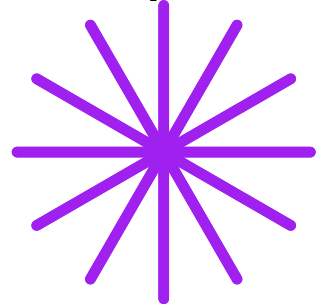
Replacing Part of a Pict

```
def spike_ring:
  overlay(& (for List (i in 0..spikes):
    def angle = 2 * math.pi * i / spikes
    line(~dx: spike_len * math.cos(angle),
      ~dy: spike_len * math.sin(angle),
      ~line: "purple", ~line_width: 4)))

def urchin = overlay(spike_ring, circle(~size: 30, ~fill: "purple"))
```

Replacing Part of a Pict

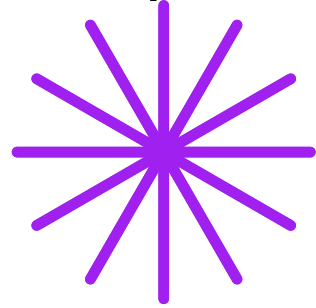
```
def spike_ring:
```



```
def urchin = overlay(spike_ring, circle(~size: 30, ~fill: "purple"))
```


Replacing Part of a Pict

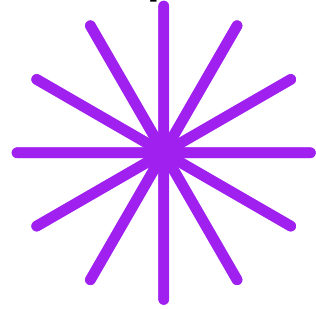
```
def spike_ring:
```

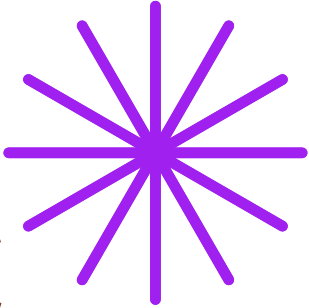


```
def urchin = overlay(spike_ring, circle(~size: 30, ~fill: "purple"))
```

Replacing Part of a Pict

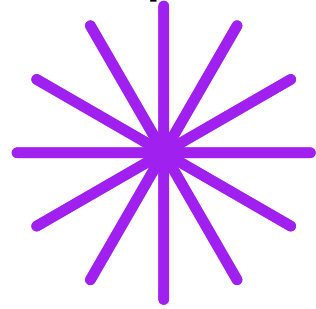
```
def spike_ring:
```

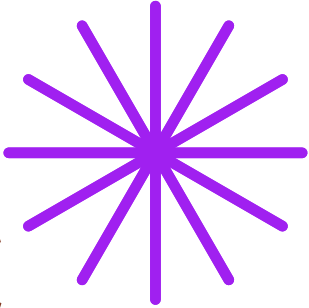


```
def urchin = overlay(, circle(~size: 30, ~fill: "purple"))
```


Replacing Part of a Pict

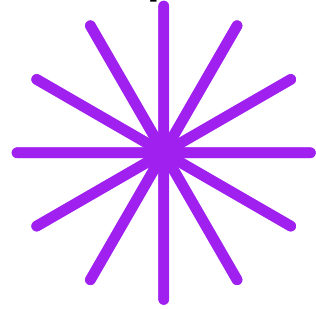
```
def spike_ring:
```

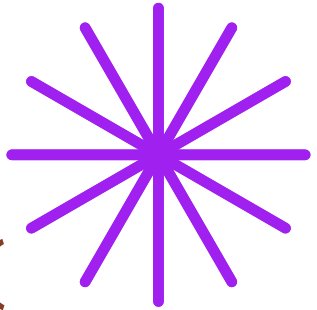


```
def urchin = overlay(, circle(~size: 30, ~fill: "purple"))
```

Replacing Part of a Pict

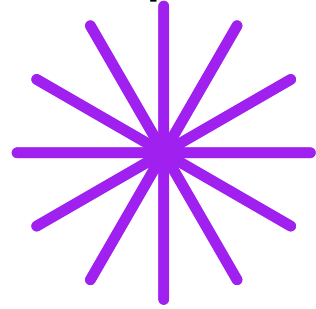
```
def spike_ring:
```

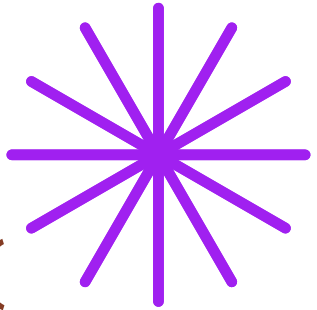


```
def urchin = overlay( , )
```

Replacing Part of a Pict

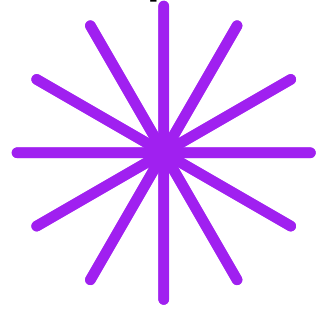
```
def spike_ring:
```



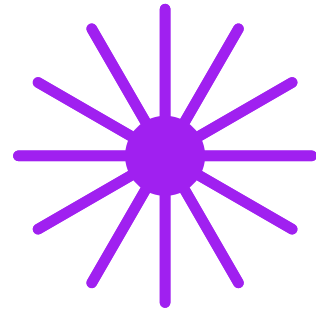
```
def urchin = overlay( , )
```

Replacing Part of a Pict

```
def spike_ring:
```

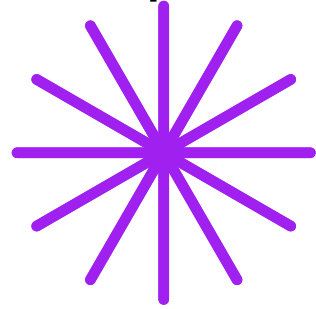


```
def urchin =
```

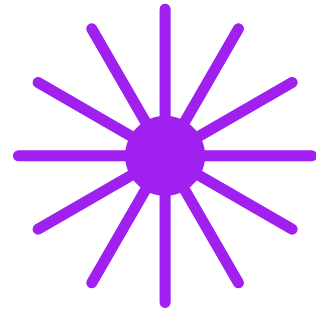


Replacing Part of a Pict

```
def spike_ring:
```



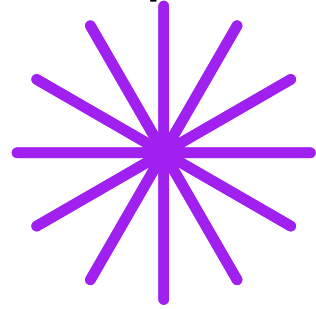
```
def urchin =
```



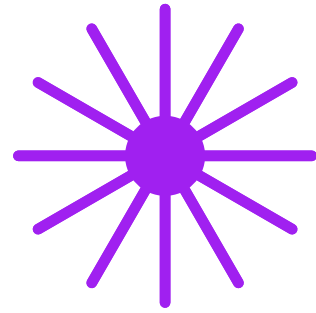
```
def row = beside(~sep: 24, , , )
```

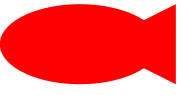
Replacing Part of a Pict

```
def spike_ring:
```



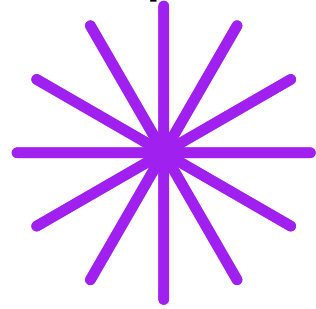
```
def urchin =
```



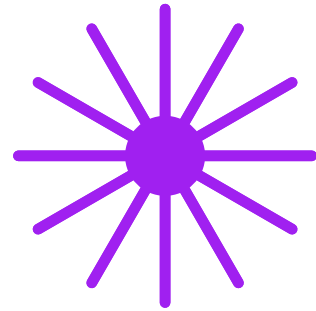
```
def row = beside(~sep: 24, , , )
```


Replacing Part of a Pict

```
def spike_ring:
```



```
def urchin =
```

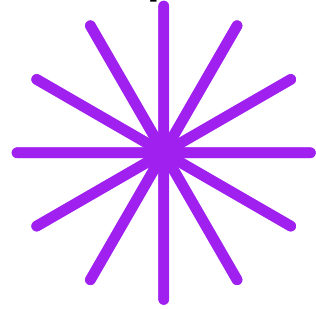


```
def row =
```

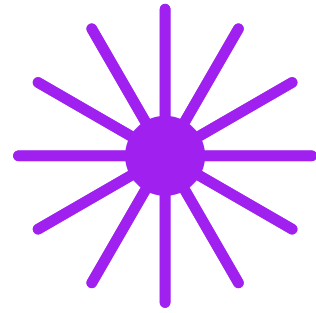


Replacing Part of a Pict

```
def spike_ring:
```



```
def urchin =
```



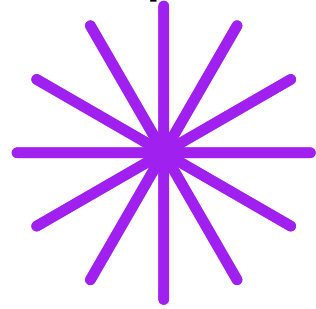
```
def row =
```



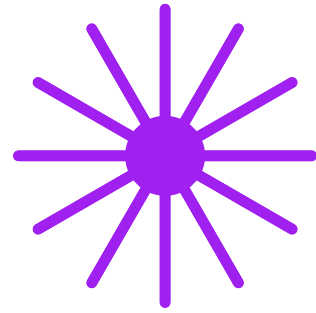
```
row.replace(, urchin)
```


Replacing Part of a Pict

```
def spike_ring:
```

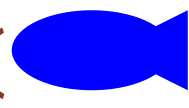


```
def urchin =
```



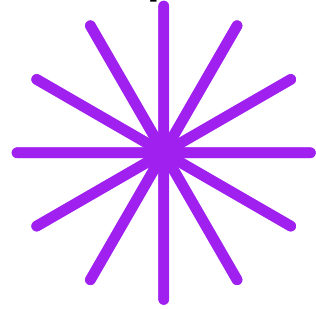
```
def row =
```



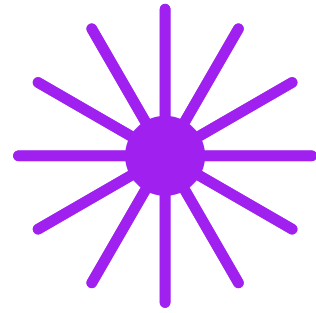
```
row.replace(, urchin)
```

Replacing Part of a Pict

```
def spike_ring:
```

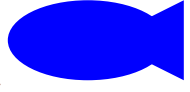


```
def urchin =
```



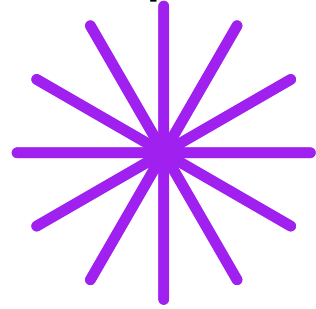
```
def row =
```



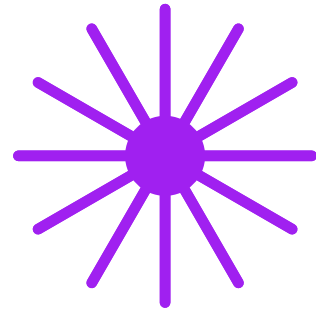
```
row.replace(, urchin)
```

Replacing Part of a Pict

```
def spike_ring:
```



```
def urchin =
```



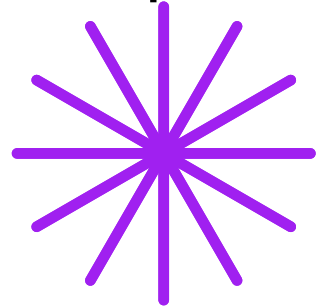
```
def row =
```



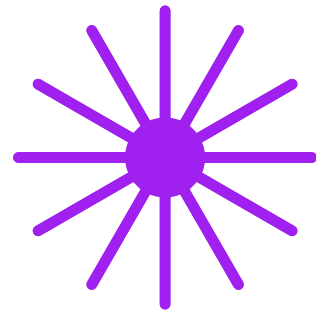
```
row.replace(, urchin)
```

Replacing Part of a Pict

```
def spike_ring:
```



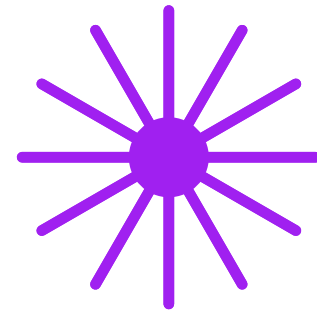
```
def urchin =
```



```
def row =
```



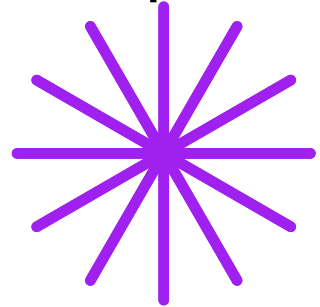
```
  .replace(,
```



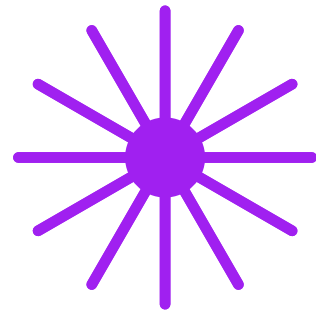
```
)
```

Replacing Part of a Pict

```
def spike_ring:
```

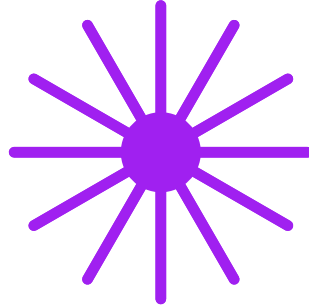


```
def urchin =
```



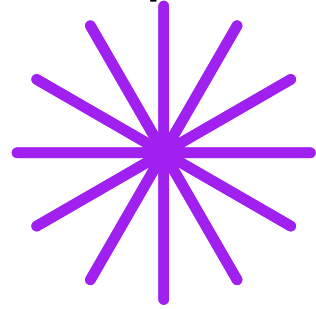
```
def row =
```



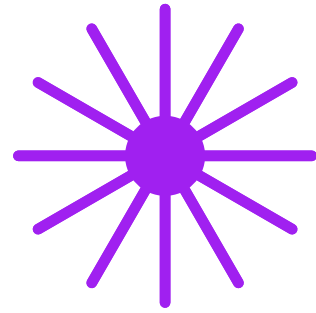
```
row.replace(, )
```

Replacing Part of a Pict

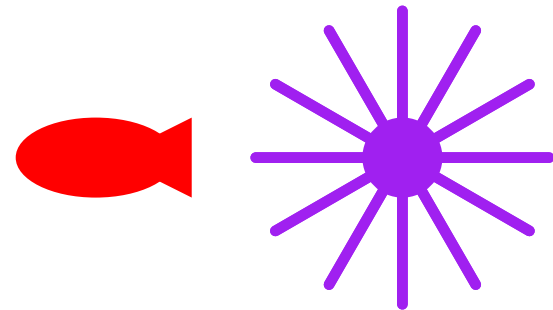
```
def spike_ring:
```



```
def urchin =
```



```
def row =
```



Replacement through User-Defined Functions

Replacement through User-Defined Functions

```
fun tank(p :: Pict):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```


Replacement through User-Defined Functions

```
fun tank(p :: Pict):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

```
tank(red_fish).replace(red_fish, urchin)
```

Replacement through User-Defined Functions

```
fun tank(p :: Pict):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

```
tank(red_fish).replace(red_fish, urchin)
```

Replacement through User-Defined Functions

```
fun tank(p :: Pict):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

```
tank().replace(red_fish, urchin)
```

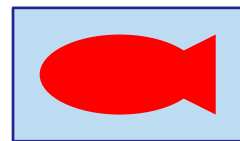
Replacement through User-Defined Functions

```
fun tank(p :: Pict):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

```
tank().replace(red_fish, urchin)
```

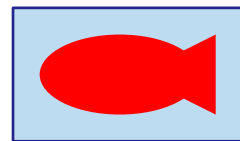
Replacement through User-Defined Functions

```
fun tank(p :: Pict):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

.replace(red_fish, urchin)

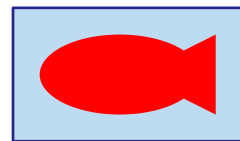
Replacement through User-Defined Functions

```
fun tank(p :: Pict):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

.replace(red_fish, urchin)

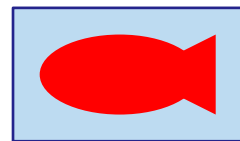
Replacement through User-Defined Functions

```
fun tank(p :: Pict):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

```
.replace(, urchin)
```

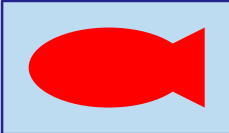
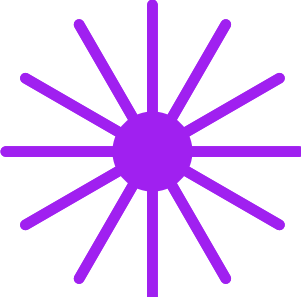
Replacement through User-Defined Functions

```
fun tank(p :: Pict):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

```
.replace(, urchin)
```

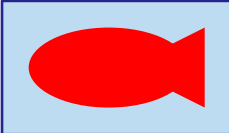
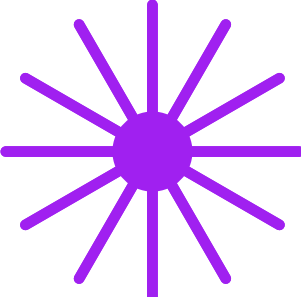

Replacement through User-Defined Functions

```
fun tank(p :: Pict):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

.replace(, )

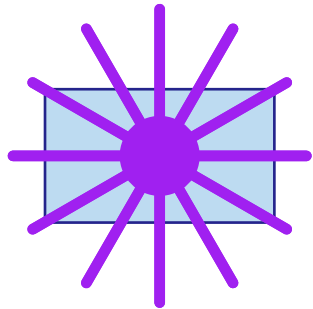
Replacement through User-Defined Functions

```
fun tank(p :: Pict):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

.replace(, )

Replacement through User-Defined Functions

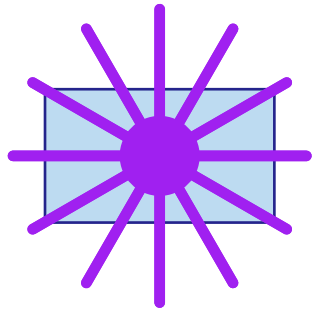
```
fun tank(p :: Pict):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```



Replacement through User-Defined Functions

still the fish's width

```
fun tank(p :: Pict):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
    ~line: "navy", ~fill: water),  
    p)
```



Replacement through User-Defined Functions

```
fun tank(p :: Pict):  
  overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                    ~line: "navy", ~fill: water),  
          p)
```

```
tank(red_fish).replace(red_fish, urchin)
```

Replacement through User-Defined Functions

```
fun tank(p :: Pict):  
  as_rebuildable (p):  
    overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                      ~line: "navy", ~fill: water),  
            p)
```

```
tank(red_fish).replace(red_fish, urchin)
```

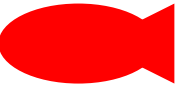

Replacement through User-Defined Functions

```
fun tank(p :: Pict):  
  as_rebuildable (p):  
    overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                      ~line: "navy", ~fill: water),  
            p)
```

```
tank(red_fish).replace(red_fish, urchin)
```

Replacement through User-Defined Functions

```
fun tank(p :: Pict):  
  as_rebuildable (p):  
    overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                      ~line: "navy", ~fill: water),  
            p)
```

```
tank().replace(red_fish, urchin)
```

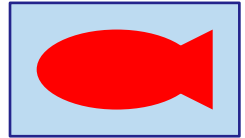

Replacement through User-Defined Functions

```
fun tank(p :: Pict):  
  as_rebuildable (p):  
    overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                      ~line: "navy", ~fill: water),  
            p)
```

```
tank().replace(red_fish, urchin)
```

Replacement through User-Defined Functions

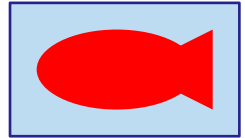
```
fun tank(p :: Pict):  
  as_rebuildable (p):  
    overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                      ~line: "navy", ~fill: water),  
            p)
```



```
.replace(red_fish, urchin)
```

Replacement through User-Defined Functions

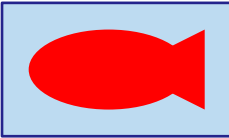
```
fun tank(p :: Pict):  
  as_rebuildable (p):  
    overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                      ~line: "navy", ~fill: water),  
            p)
```



```
.replace(red_fish, urchin)
```

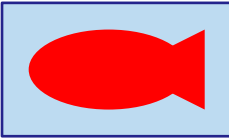
Replacement through User-Defined Functions

```
fun tank(p :: Pict):  
  as_rebuildable (p):  
    overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                      ~line: "navy", ~fill: water),  
            p)
```

.replace(, urchin)

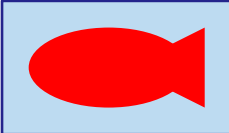
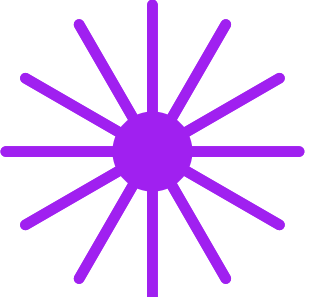
Replacement through User-Defined Functions

```
fun tank(p :: Pict):  
  as_rebuildable (p):  
    overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                      ~line: "navy", ~fill: water),  
            p)
```

.replace(, urchin)

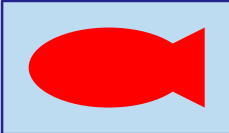
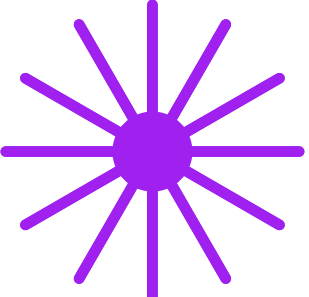
Replacement through User-Defined Functions

```
fun tank(p :: Pict):  
  as_rebuildable (p):  
    overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                      ~line: "navy", ~fill: water),  
            p)
```

.replace(, )

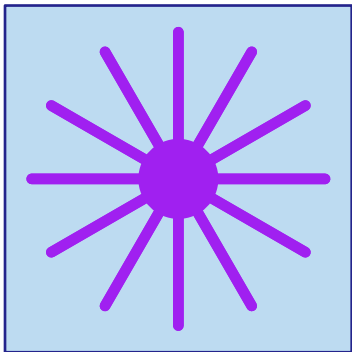
Replacement through User-Defined Functions

```
fun tank(p :: Pict):  
  as_rebuildable (p):  
    overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                      ~line: "navy", ~fill: water),  
            p)
```

.replace(, )

Replacement through User-Defined Functions

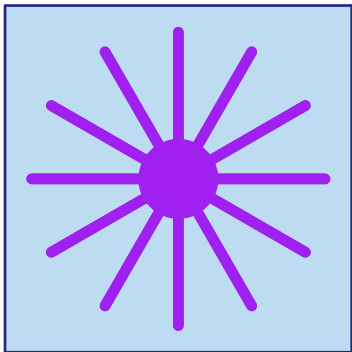
```
fun tank(p :: Pict):  
  as_rebuildable (p):  
    overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                      ~line: "navy", ~fill: water),  
            p)
```



Replacement through User-Defined Functions

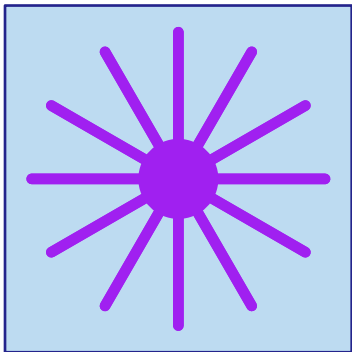
```
fun tank(p :: Pict):  
  as_rebuildable (p):  
    overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
                      ~line: "navy", ~fill: water),  
            p)
```

a Rhombus macro



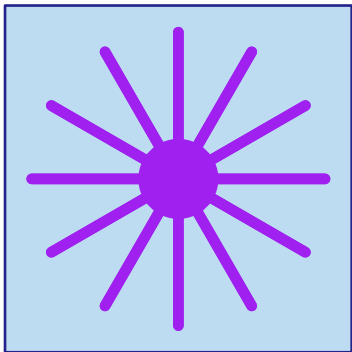
Replacement through User-Defined Functions

```
fun tank(p :: Pict):  
  rebuildable(~deps: [p],  
    fun (~deps: [p]):  
      overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
        ~line: "navy", ~fill: water),  
        p)))
```



Replacement through User-Defined Functions

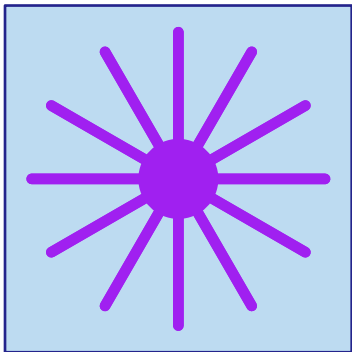
```
fun tank(p :: Pict):  
  rebuildable(~deps: [p],  
    fun (~deps: [p])> specify dependencies  
      overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
        ~line: "navy", ~fill: water),  
        p)))
```



Replacement through User-Defined Functions

initial dependencies

```
fun tank(p :: Pict):  
  rebuildable(~deps: [p],  
    fun (~deps: [p])< specify dependencies  
      overlay(rectangle(~width: p.width + 20, ~height: p.height + 20,  
        ~line: "navy", ~fill: water),  
        p))
```



More Examples

More Examples

```
def spinning_fish:
    animate(~extent: 2,
            fun (n):
                fish("purple", 30).rotate(n * 2 * math.pi))

row.replace(, spinning_fish)
```


More Examples

```
def spinning_fish:
    animate(~extent: 2,
            fun (n):
                fish("purple", 30).rotate(n * 2 * math.pi))

row.replace(, spinning_fish)
```

More Examples

```
def spinning_fish:
```



```
row.replace(, spinning_fish)
```


More Examples

```
def spinning_fish:
```



```
row.replace(, spinning_fish)
```

More Examples

```
def spinning_fish:
```



```
  .replace(, spinning_fish)
```

More Examples

```
def spinning_fish:
```



```
  .replace(, spinning_fish)
```

More Examples

```
def spinning_fish:
```



```
      .replace(, )
```

More Examples

```
def spinning_fish:
```



```
      .replace(, )
```

More Examples

```
def spinning_fish:
```



More Examples

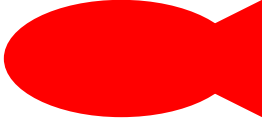
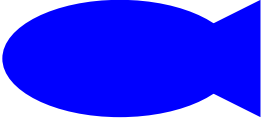
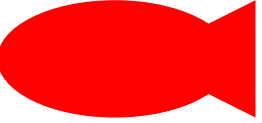
```
def spinning_fish:
```



More Examples

```
def spinning_fish:
```



```
magic_move(beside(~sep: 14, , , , ) ,  
           stack(~sep: 10,  
                beside(~sep: 14, , ) ,  
                beside(~sep: 14, , 

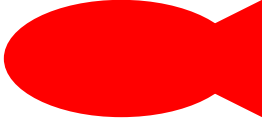
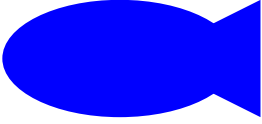
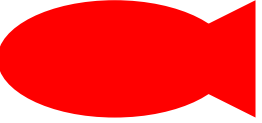
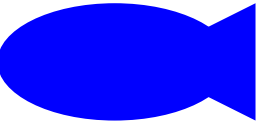
16


```


More Examples

```
def spinning_fish:
```

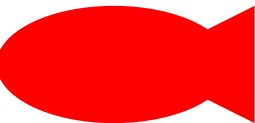
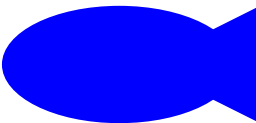
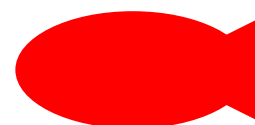
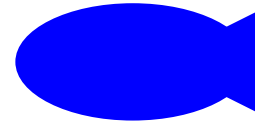


```
magic_move(beside(~sep: 14, , , , ),  
           stack(~sep: 10,  
                beside(~sep: 14, , ) ,  
                beside(~sep: 14, , )))
```

More Examples

```
def spinning_fish:
```

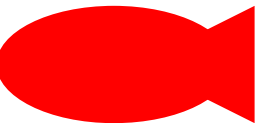
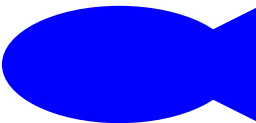
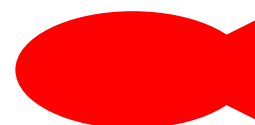
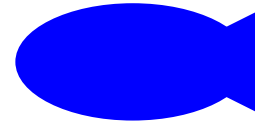


```
magic_move(   ,  
  stack(~sep: 10,  
    beside(~sep: 14, , ) ,  
    beside(~sep: 14, , )))
```

More Examples

```
def spinning_fish:
```

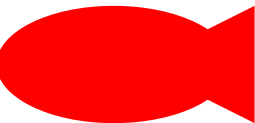
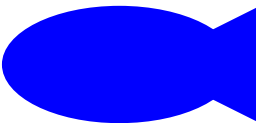
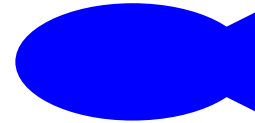


```
magic_move(   ,  
    stack(~sep: 10,  
        beside(~sep: 14, , ) ,  
        beside(~sep: 14, , )))
```

More Examples

```
def spinning_fish:
```

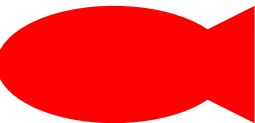
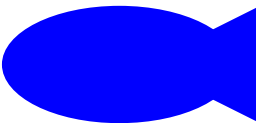
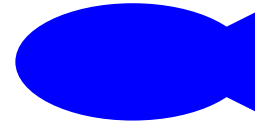


```
magic_move(   ,  
  stack(~sep: 10,  
     ,  
    beside(~sep: 14, , )))
```

More Examples

```
def spinning_fish:
```

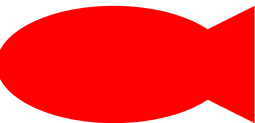
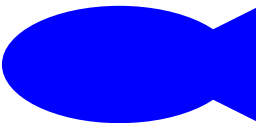
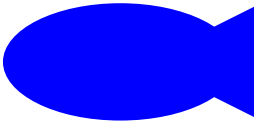


```
magic_move(   ,  
  stack(~sep: 10,  
     ,  
    beside(~sep: 14, , )))
```

More Examples

```
def spinning_fish:
```

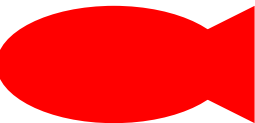
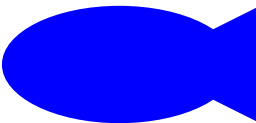
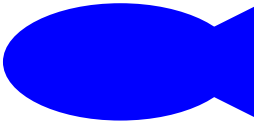


```
magic_move(   ,  
stack(~sep: ,  
       ,  
       )))
```


More Examples

```
def spinning_fish:
```



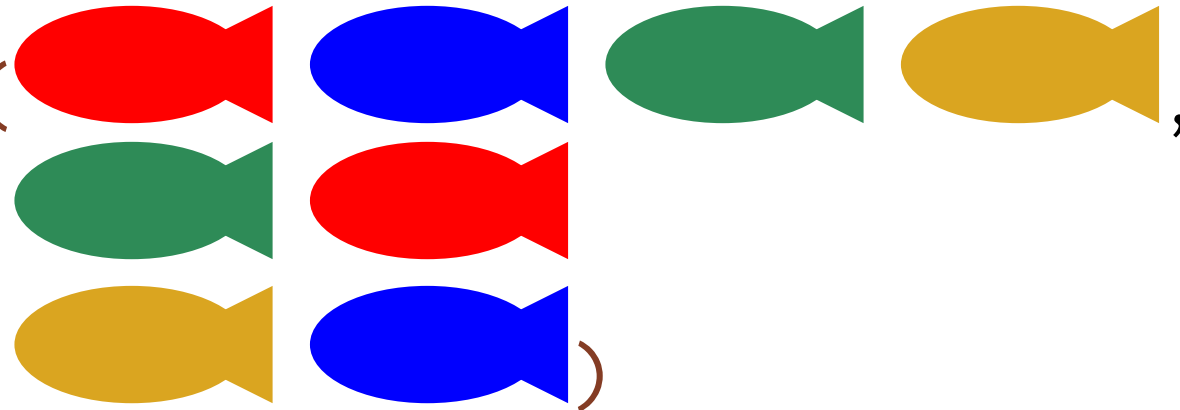
```
magic_move(   ,  
stack(~sep: 10,  
       ,  
       ))
```

More Examples

```
def spinning_fish:
```



```
magic_move(
```



```
)
```

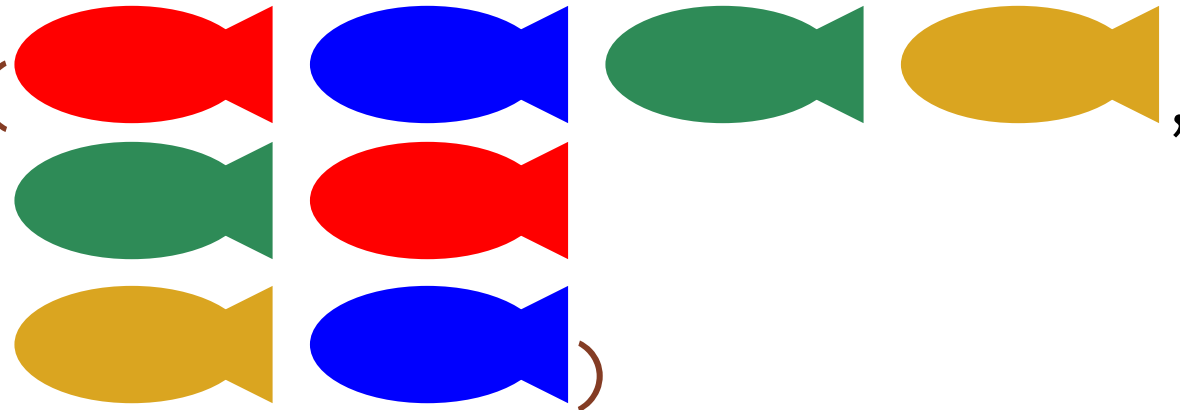
```
,
```


More Examples

```
def spinning_fish:
```



```
magic_move(
```

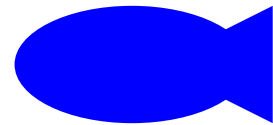
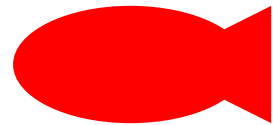


```
)
```

```
,
```

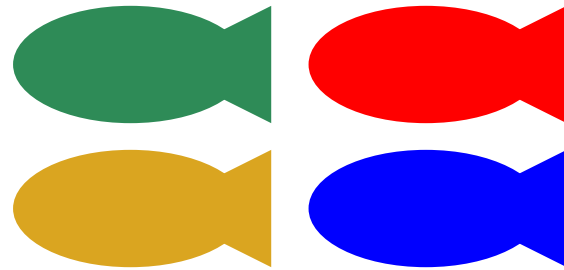
More Examples

```
def spinning_fish:
```



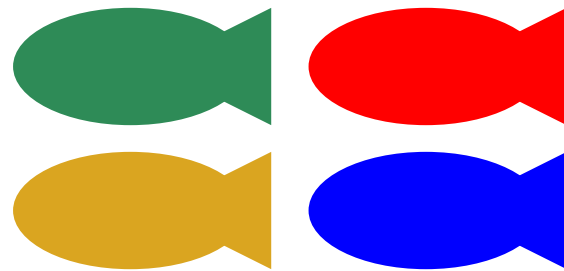
More Examples

```
def spinning_fish:
```



More Examples

```
def spinning_fish:
```



```
def seed:
```

```
    stack(,
```

```
        beside(, ))
```

```
fun level(k):
```

```
    if k == 0
```

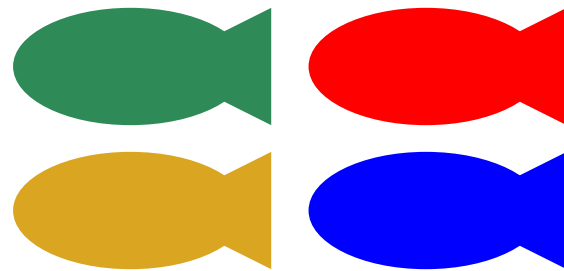
```
    | fish
```

```
    | seed.replace(fish, level(k-1))
```



More Examples

```
def spinning_fish:
```



```
def seed:
```

```
    stack(,
```

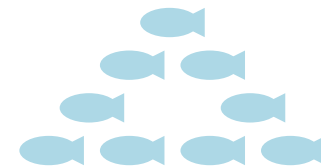
```
        beside(, ))
```

```
fun level(k):
```

```
    if k == 0
```

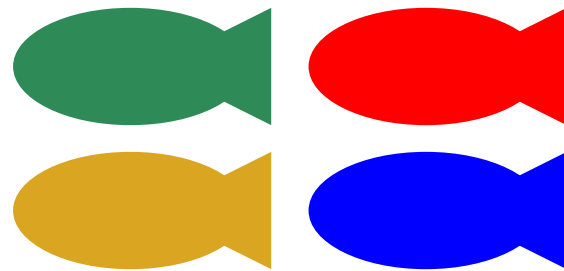
```
    | fish
```

```
    | seed.replace(fish, level(k-1))
```



More Examples

```
def spinning_fish:
```



```
def seed:
```

```
    stack(,
```

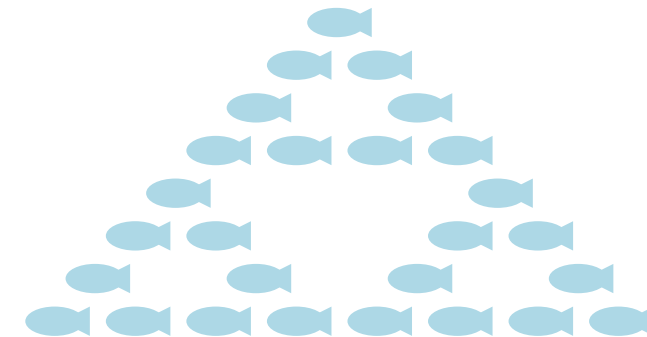
```
        beside(, ))
```

```
fun level(k):
```

```
    if k == 0
```

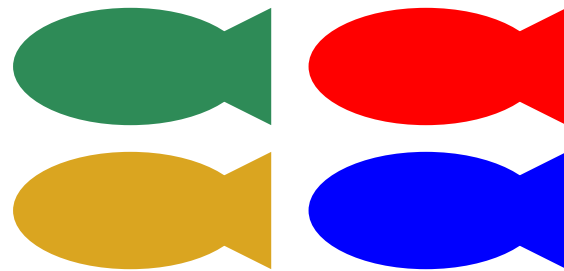
```
    | fish
```

```
    | seed.replace(fish, level(k-1))
```



More Examples

```
def spinning_fish:
```



```
def seed:
```

```
    stack(,
```

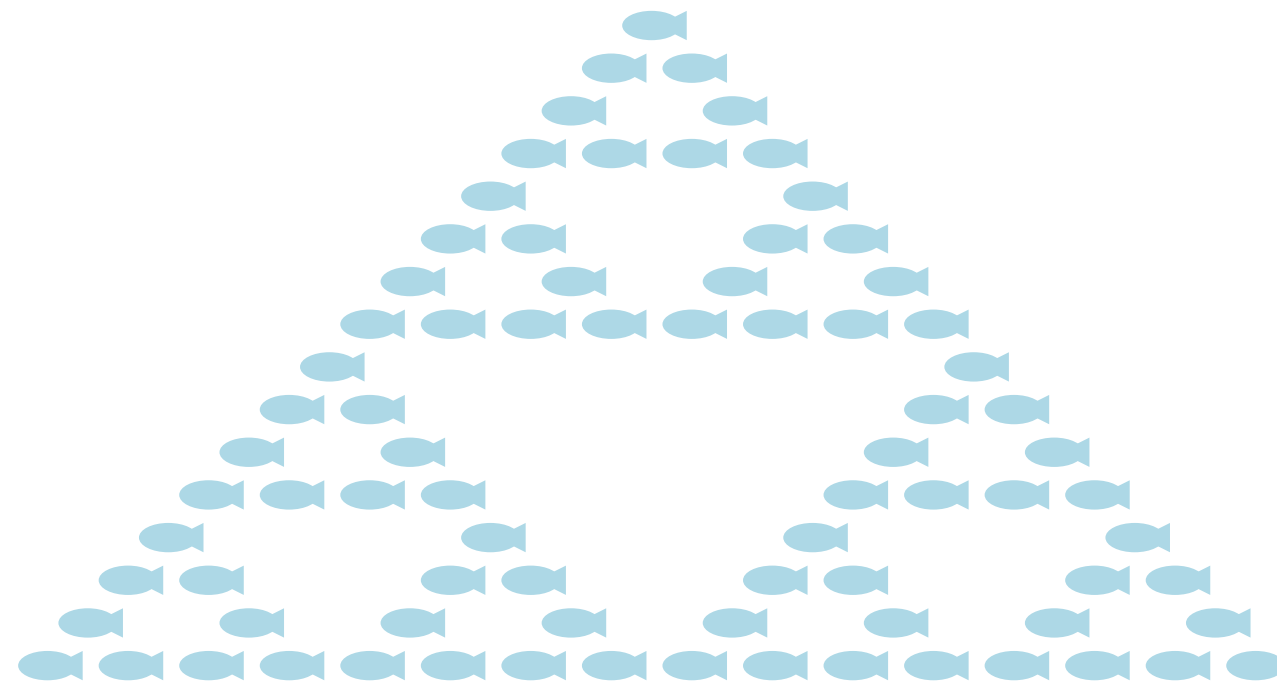
```
        beside(, ))
```

```
fun level(k):
```

```
    if k == 0
```

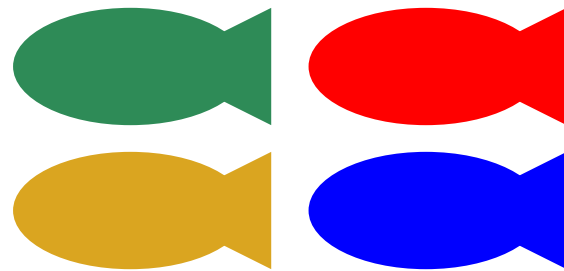
```
    | fish
```

```
    | seed.replace(fish, level(k-1))
```



More Examples

```
def spinning_fish:
```



```
def seed:
```

```
    stack(,
```

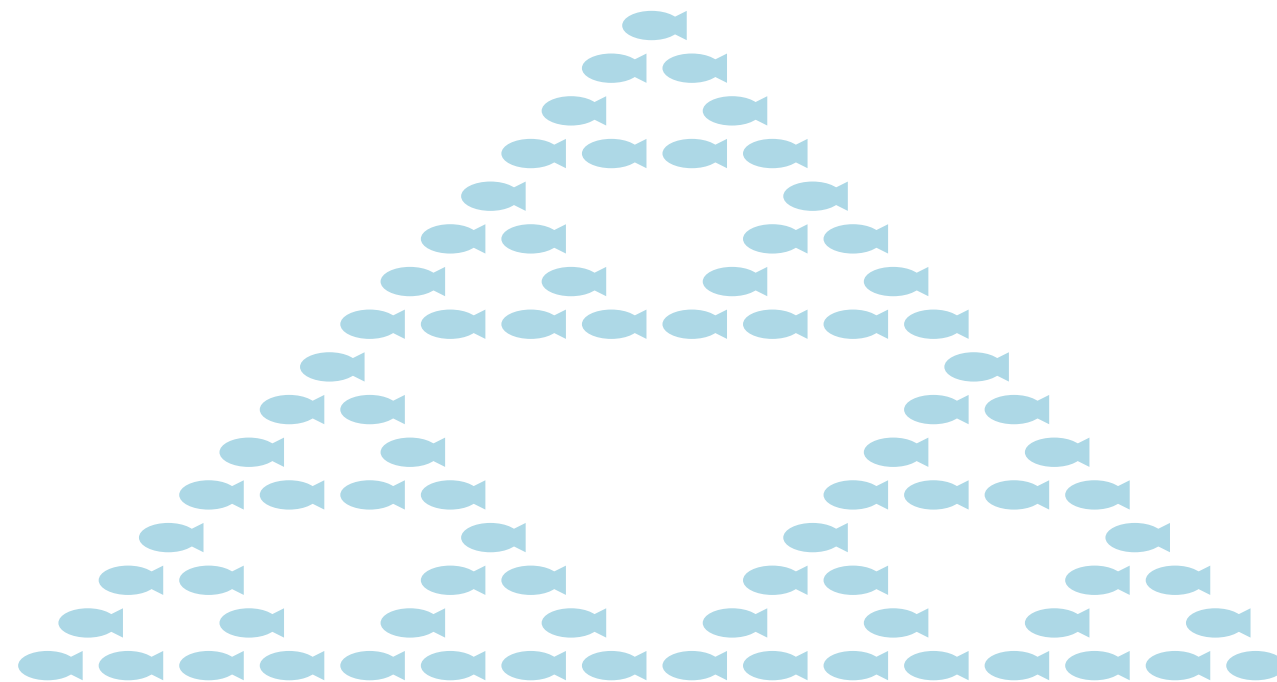
```
        beside(, ))
```

```
fun level(k):
```

```
    if k == 0
```

```
    | fish
```

```
    | seed.replace(fish, level(k-1))
```



Outline

Outline

- 1 Motivation
- 2 Making Animations Composable
- 3 Replacing Parts of Animations**
- 4 Reflection

Outline

- 1 Motivation
- 2 Making Animations Composable
- 3 Replacing Parts of Animations
- 4 Reflection

How was this slideshow implemented?

How was this slideshow implemented?

```
def [code, steps]:  
    eval_steps:  
        overlay(ellipse(~width: 100, ~height: 50, ~fill: "lightblue"),  
                ~dx: 60,  
                triangle(~size: 50, ~fill: "lightblue").rotate(1/2 * math.pi))  
  
small_steps(code, steps)
```

How was this slideshow implemented?

```
def [code, steps]:  
    eval_steps: a Rhombus macro  
    overlay(ellipse(~width: 100, ~height: 50, ~fill: "lightblue"),  
            ~dx: 60,  
            triangle(~size: 50, ~fill: "lightblue").rotate(1/2 * math.pi))  
  
small_steps(code, steps)
```


How was this slideshow implemented?

```
def [code, steps] a list of pict replacements
  eval_steps. a Rhombus macro
  overlay(ellipse(~width: 100, ~height: 50, ~fill: "lightblue"),
    ~dx: 60,
    triangle(~size: 50, ~fill: "lightblue").rotate(1/2 * math.pi))

small_steps(code, steps)
```

How was this slideshow implemented?

```
def [code, steps, eval_steps]:
    a list of pict replacements
    a Rhombus macro
    overlay(ellipse(~width: 100, ~height: 50, ~fill: "lightblue"),
            ~dx: 60,
            triangle(~size: 50, ~fill: "lightblue").rotate(1/2 * math.pi))

small_steps(code, [
    Step(ellipse(~width: 100, ~height: 50, ~fill: "lightblue"), , ),
    Step(triangle(~size: 50, ~fill: "lightblue"), , ),
    Step(math.pi, 3.141592653589793),
    Step(1/2 * math.pi, 1.5707963267948966),
    Step(triangle(~size: 50, ~fill: "lightblue").rotate(1/2 * math.pi), , ),
    Step(overlay(ellipse(~width: 100, ~height: 50, ~fill: "lightblue"),
                ~dx: 60,
                triangle(~size: 50, ~fill: "lightblue").rotate(1/2 * math.pi)), ,
    ])
]
```


How was this slideshow implemented?

```
overlay(ellipse(~width: 100,  
               ~height: 50,  
               ~fill: "lightblue"),  
~dx: 60, ~dy: 0,  
triangle(~size: 50,  
         ~fill: "lightblue")  
        .rotate(1/2 * math.pi))
```

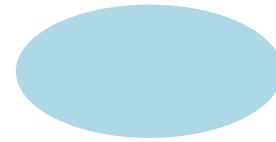
How was this slideshow implemented?

```
overlay(ellipse(~width: 100,  
               ~height: 50,  
               ~fill: "lightblue"),  
       ~dx: 60, ~dy: 0,  
       triangle(~size: 50,  
               ~fill: "lightblue")  
       .rotate(1/2 * math.pi))
```

```
ellipse(~width: 100,  
       ~height: 50,  
       ~fill: "lightblue")
```

How was this slideshow implemented?

```
overlay(ellipse(~width: 100,  
               ~height: 50,  
               ~fill: "lightblue"),  
       ~dx: 60, ~dy: 0,  
       triangle(~size: 50,  
               ~fill: "lightblue")  
       .rotate(1/2 * math.pi))
```



How was this slideshow implemented?

```
overlay(,  
    ~dx: 60, ~dy: 0,  
    triangle(~size: 50,  
        ~fill: "lightblue")  
        .rotate(1/2 * math.pi))
```

How was this slideshow implemented?

```
overlay(,  
  ~dx: 60, ~dy: 0,  
  triangle(~size: 50,  
    ~fill: "lightblue")  
    .rotate(1/2 * math.pi))
```

```
triangle(~size: 50,  
  ~fill: "lightblue")  
  .rotate(1/2 * math.pi)
```

How was this slideshow implemented?

```
overlay(,  
  ~dx: 60, ~dy: 0,  
  triangle(~size: 50,  
    ~fill: "lightblue")  
    .rotate(1/2 * math.pi))  
  
triangle(~size: 50,  
  ~fill: "lightblue")  
  .rotate(1/2 * math.pi)  
  
triangle(~size: 50,  
  ~fill: "lightblue")
```


How was this slideshow implemented?

```
overlay(,  
  ~dx: 60, ~dy: 0,  
  triangle(~size: 50,  
    ~fill: "lightblue")  
    .rotate(1/2 * math.pi))  
  
triangle(~size: 50,  
  ~fill: "lightblue")  
  .rotate(1/2 * math.pi)
```



How was this slideshow implemented?

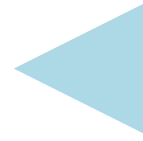
```
overlay(,  
  ~dx: 60, ~dy: 0,  
  triangle(~size: 50,  
    ~fill: "lightblue")  
    .rotate(1/2 * math.pi))
```



```
.rotate(1/2 * math.pi)
```


How was this slideshow implemented?

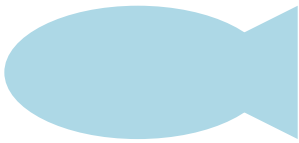
```
overlay(,  
  ~dx: 60, ~dy: 0,  
  triangle(~size: 50,  
    ~fill: "lightblue")  
    .rotate(1/2 * math.pi))
```



How was this slideshow implemented?

```
overlay(,  
  ~dx: 60, ~dy: 0,  
  )
```

How was this slideshow implemented?



Shallow vs Deep Embeddings

Shallow vs Deep Embeddings

Shallow

```
fun add(a, b): a + b
```

```
fun sub(a, b): a - b
```

```
add(1, sub(5, 2))
```

Shallow vs Deep Embeddings

Shallow

```
fun add(a, b): a + b
```

```
fun sub(a, b): a - b
```

```
add(1, sub(5, 2))
```


Shallow vs Deep Embeddings

Shallow

```
fun add(a, b): a + b
```

```
fun sub(a, b): a - b
```

```
add(1, 5 - 2)
```

Shallow vs Deep Embeddings

Shallow

```
fun add(a, b): a + b
```

```
fun sub(a, b): a - b
```

```
add(1, 5 - 2)
```


Shallow vs Deep Embeddings

Shallow

```
fun add(a, b): a + b
```

```
fun sub(a, b): a - b
```

```
add(1, 3)
```

Shallow vs Deep Embeddings

Shallow

```
fun add(a, b): a + b
```

```
fun sub(a, b): a - b
```

```
add(1, 3)
```

Shallow vs Deep Embeddings

Shallow

```
fun add(a, b): a + b
```

```
fun sub(a, b): a - b
```

```
1 + 3
```

Shallow vs Deep Embeddings

Shallow

```
fun add(a, b): a + b
```

```
fun sub(a, b): a - b
```

```
1 + 3
```

Shallow vs Deep Embeddings

Shallow

```
fun add(a, b): a + b  
fun sub(a, b): a - b
```

4

Shallow vs Deep Embeddings

Shallow

```
fun add(a, b): a + b
fun sub(a, b): a - b
```

4

Deep

```
class Add(a, b)
class Sub(a, b)
```

```
fun eval(e):
  match e
  | Add(a, b): eval(a) + eval(b)
  | Sub(a, b): eval(a) - eval(b)
  | ~else: e
```

```
eval(
  Add(1, Sub(5, 2)))
```


Shallow vs Deep Embeddings

Shallow

```
fun add(a, b): a + b
fun sub(a, b): a - b
```

4

Deep

```
class Add(a, b)
class Sub(a, b)
```

```
fun eval(e):
  match e
  | Add(a, b): eval(a) + eval(b)
  | Sub(a, b): eval(a) - eval(b)
  | ~else: e
```

```
eval(
  Add(1, Sub(5, 2)))
```

Shallow vs Deep Embeddings

Shallow

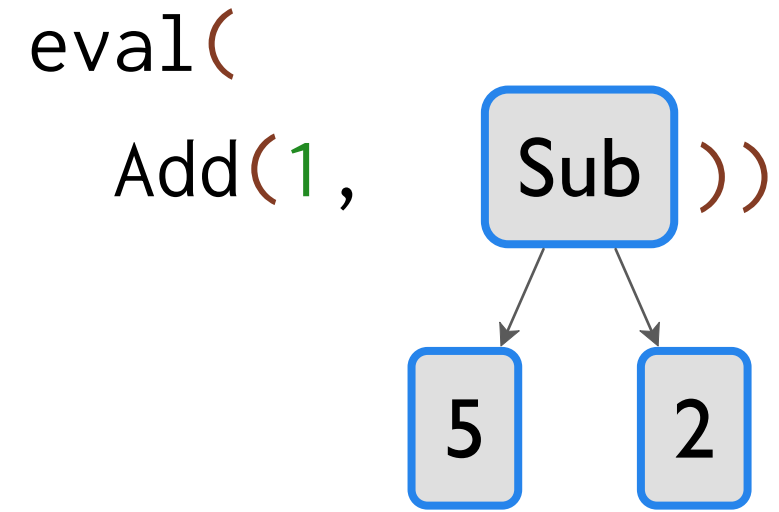
```
fun add(a, b): a + b  
fun sub(a, b): a - b
```

4

Deep

```
class Add(a, b)  
class Sub(a, b)
```

```
fun eval(e):  
  match e  
  | Add(a, b): eval(a) + eval(b)  
  | Sub(a, b): eval(a) - eval(b)  
  | ~else: e
```



Shallow vs Deep Embeddings

Shallow

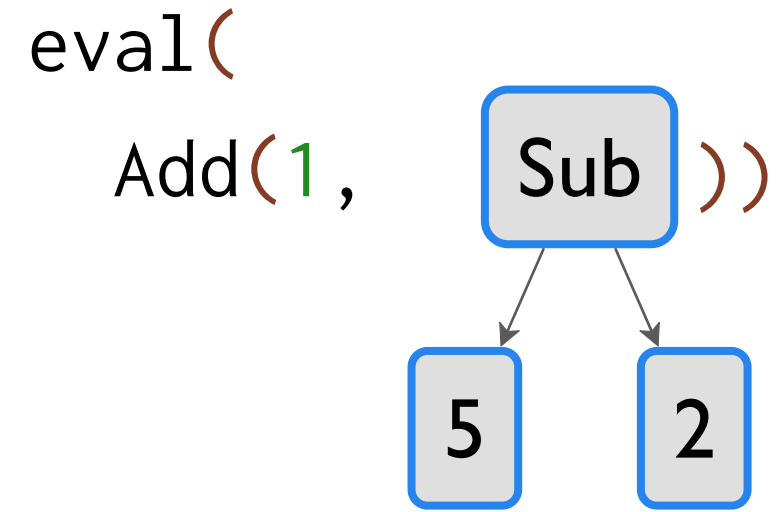
```
fun add(a, b): a + b  
fun sub(a, b): a - b
```

4

Deep

```
class Add(a, b)  
class Sub(a, b)
```

```
fun eval(e):  
  match e  
  | Add(a, b): eval(a) + eval(b)  
  | Sub(a, b): eval(a) - eval(b)  
  | ~else: e
```



Shallow vs Deep Embeddings

Shallow

```
fun add(a, b): a + b
fun sub(a, b): a - b
```

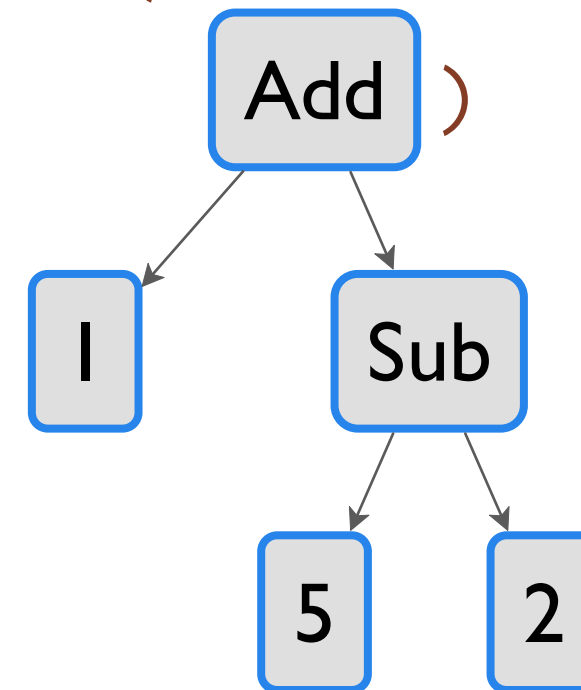
4

Deep

```
class Add(a, b)
class Sub(a, b)
```

```
fun eval(e):
  match e
  | Add(a, b): eval(a) + eval(b)
  | Sub(a, b): eval(a) - eval(b)
  | ~else: e
```

eval(



Shallow vs Deep Embeddings

Shallow

```
fun add(a, b): a + b
fun sub(a, b): a - b
```

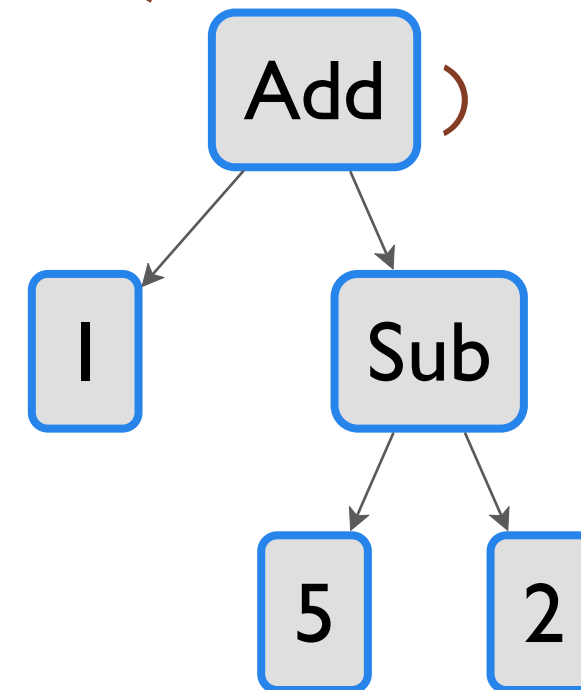
4

Deep

```
class Add(a, b)
class Sub(a, b)
```

```
fun eval(e):
  match e
  | Add(a, b): eval(a) + eval(b)
  | Sub(a, b): eval(a) - eval(b)
  | ~else: e
```

eval(



Shallow vs Deep Embeddings

Shallow

```
fun add(a, b): a + b  
fun sub(a, b): a - b
```

4

Deep

```
class Add(a, b)  
class Sub(a, b)
```

```
fun eval(e):  
  match e  
  | Add(a, b): eval(a) + eval(b)  
  | Sub(a, b): eval(a) - eval(b)  
  | ~else: e
```

4

What about Pict?

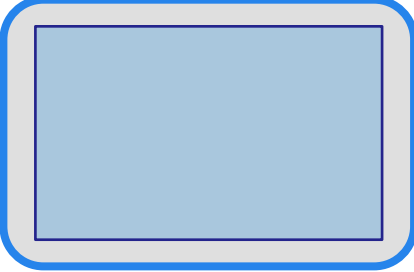
What about Pict?

```
overlay(rectangle(~width: 130, ~height: 80, ~line: "navy", ~fill: water),  
        ~dx: 36, ~dy: 28,  
        overlay(ellipse(~width: 60, ~height: 30, ~fill: "red"),  
                ~dx: 36,  
                triangle(~size: 30, ~fill: "red").rotate(1/2 * math.pi)))
```

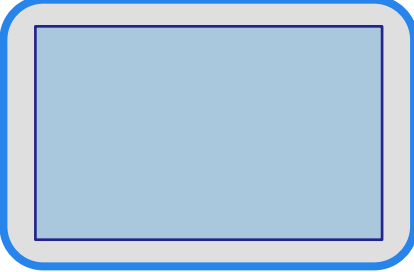

What about Pict?

```
overlay(rectangle(~width: 130, ~height: 80, ~line: "navy", ~fill: water),  
        ~dx: 36, ~dy: 28,  
        overlay(ellipse(~width: 60, ~height: 30, ~fill: "red"),  
                ~dx: 36,  
                triangle(~size: 30, ~fill: "red").rotate(1/2 * math.pi)))
```

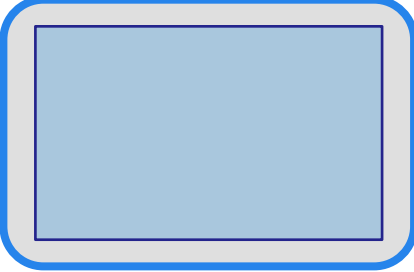
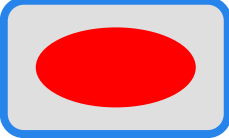
What about Pict?

```
overlay(,  
  ~dx: 36, ~dy: 28,  
  overlay(ellipse(~width: 60, ~height: 30, ~fill: "red"),  
    ~dx: 36,  
    triangle(~size: 30, ~fill: "red").rotate(1/2 * math.pi)))
```

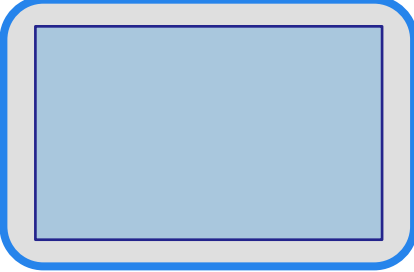
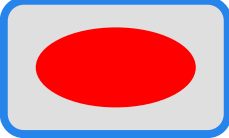

What about Pict?

```
overlay(,  
  ~dx: 36, ~dy: 28,  
  overlay(ellipse(~width: 60, ~height: 30, ~fill: "red"),  
    ~dx: 36,  
    triangle(~size: 30, ~fill: "red").rotate(1/2 * math.pi)))
```

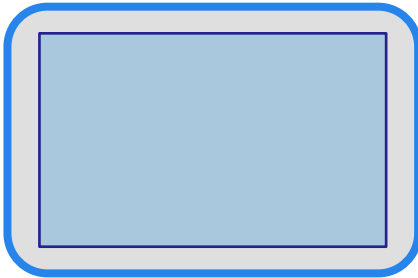
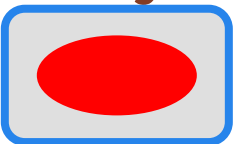
What about Pict?

```
overlay(,  
  ~dx: 36, ~dy: 28,  
  overlay(,  
    ~dx: 36,  
    triangle(~size: 30, ~fill: "red").rotate(1/2 * math.pi)))
```

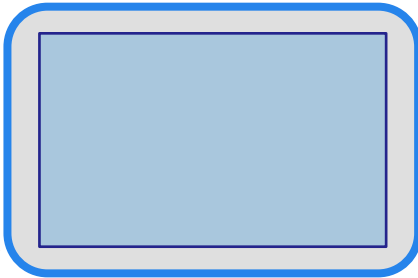
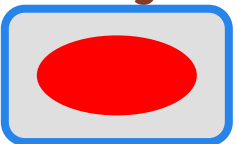
What about Pict?

```
overlay(,  
  ~dx: 36, ~dy: 28,  
  overlay(,  
    ~dx: 36,  
    triangle(~size: 30, ~fill: "red").rotate(1/2 * math.pi)))
```

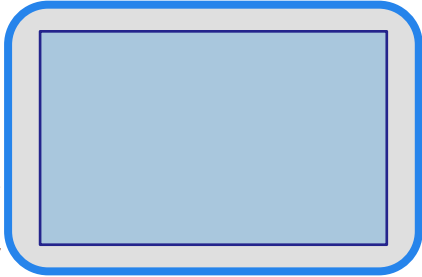
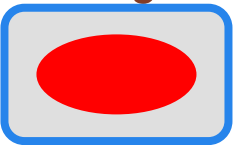
What about Pict?

```
overlay(      ~dx: 36, ~dy: 28,  
      overlay(            ~dx: 36,  
            .rotate(1/2 * math.pi)))
```

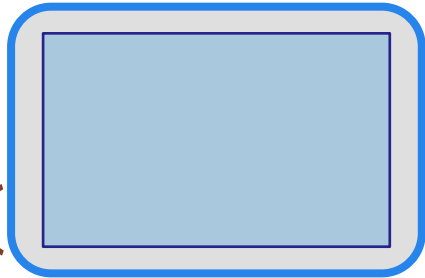
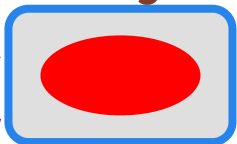
What about Pict?

```
overlay(      ~dx: 36, ~dy: 28,  
      overlay(            ~dx: 36,  
            .rotate(1/2 * math.pi)))
```

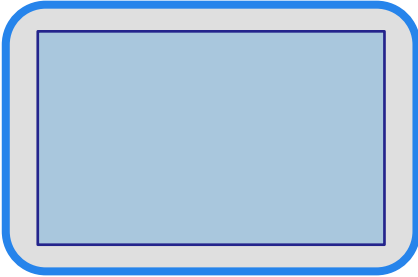
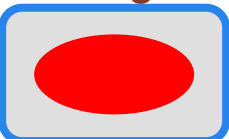
What about Pict?

```
overlay(      ~dx: 36, ~dy: 28,  
      overlay(            ~dx: 36,  
            .rotate(1/2 * 3.141592653589793)))
```

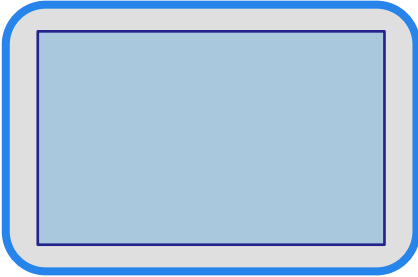
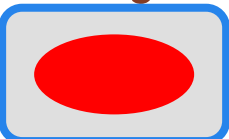

What about Pict?

```
overlay(      ~dx: 36, ~dy: 28,  
      overlay(            ~dx: 36,  
            .rotate(1/2 * 3.141592653589793)))
```

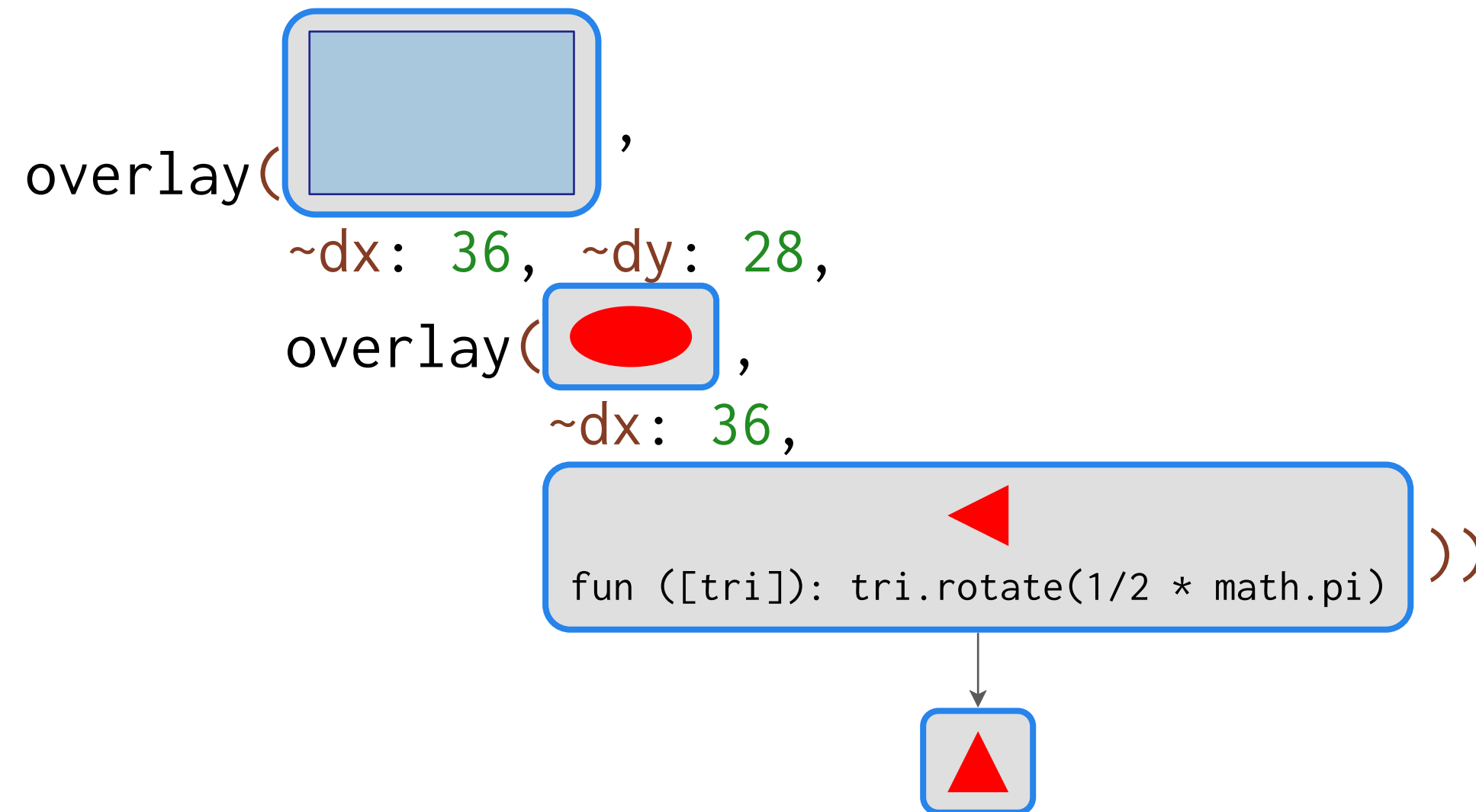
What about Pict?

```
overlay(,  
~dx: 36, ~dy: 28,  
overlay(,  
~dx: 36,  
.rotate(1.5707963267948966)))
```

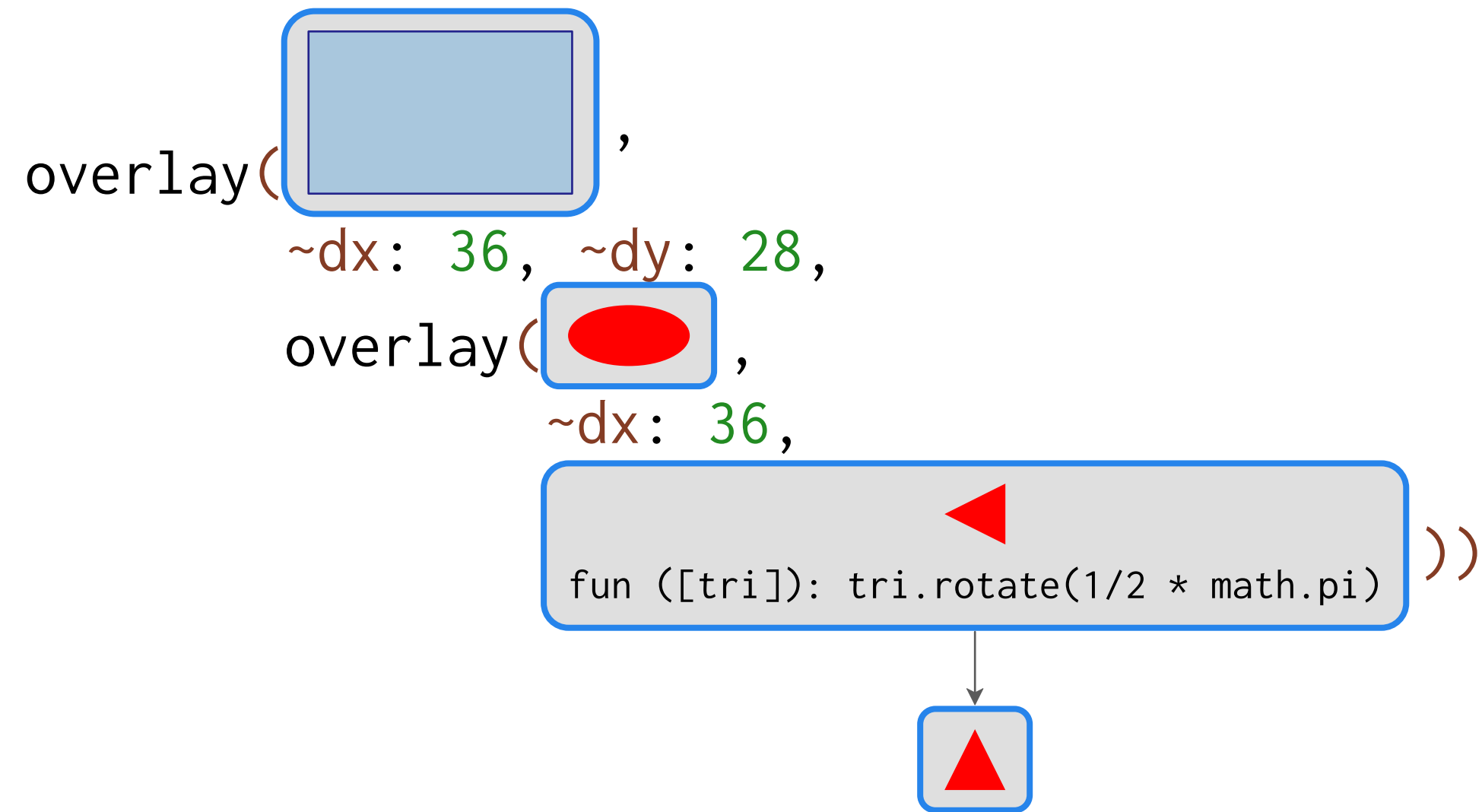

What about Pict?

```
overlay(      ~dx: 36, ~dy: 28,  
      overlay(            ~dx: 36,  
            .rotate(1.5707963267948966)))
```

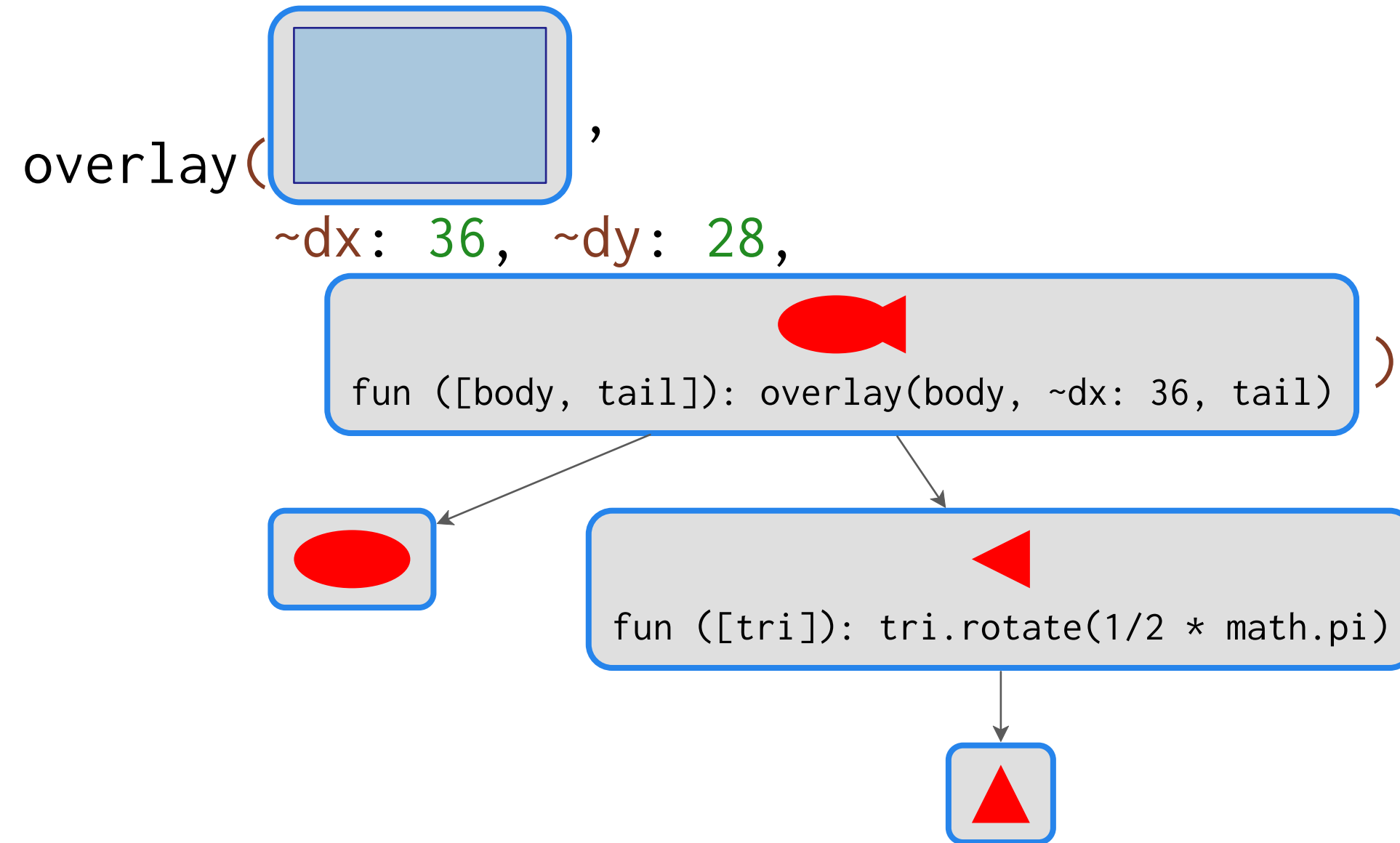
What about Pict?



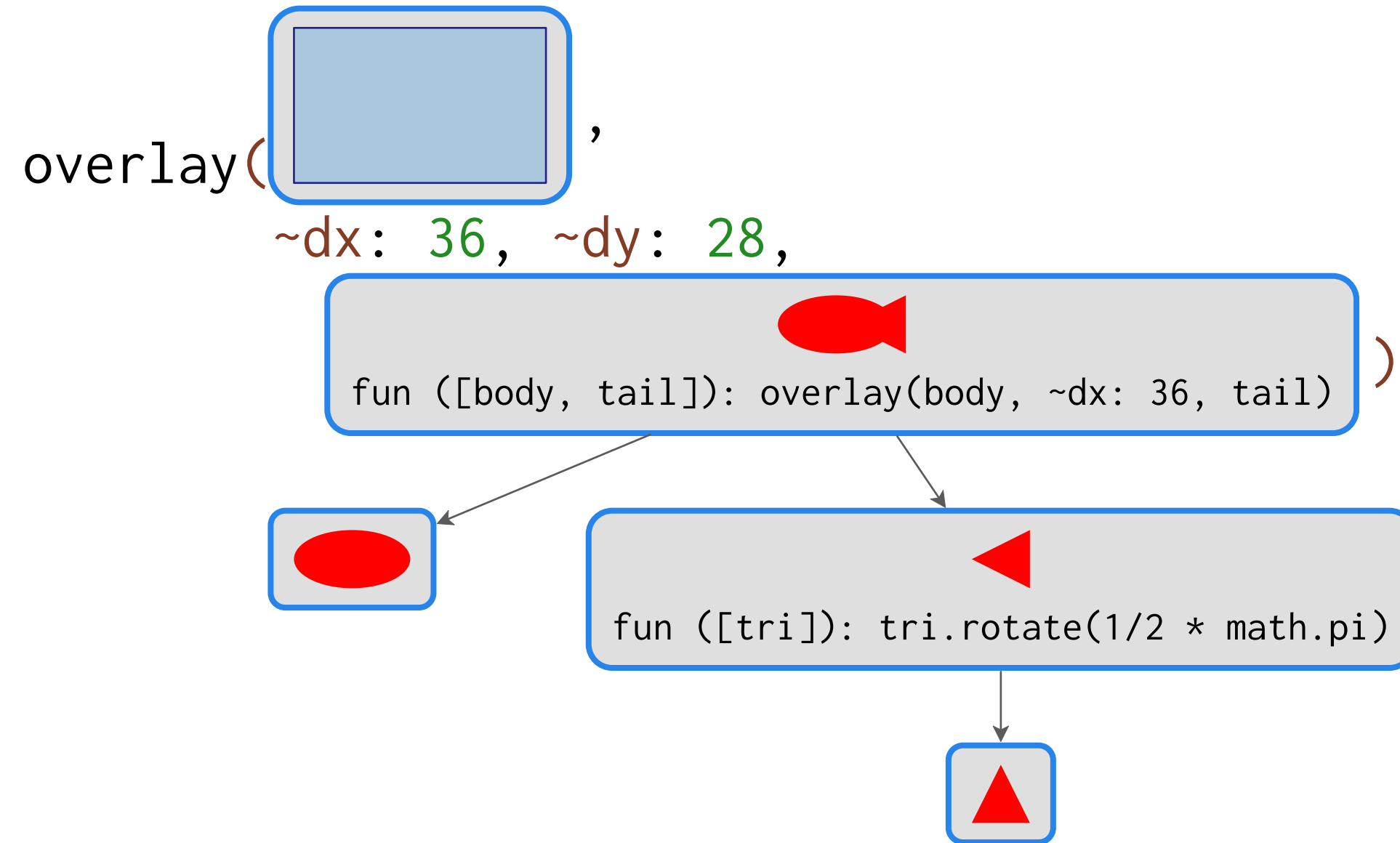
What about Pict?



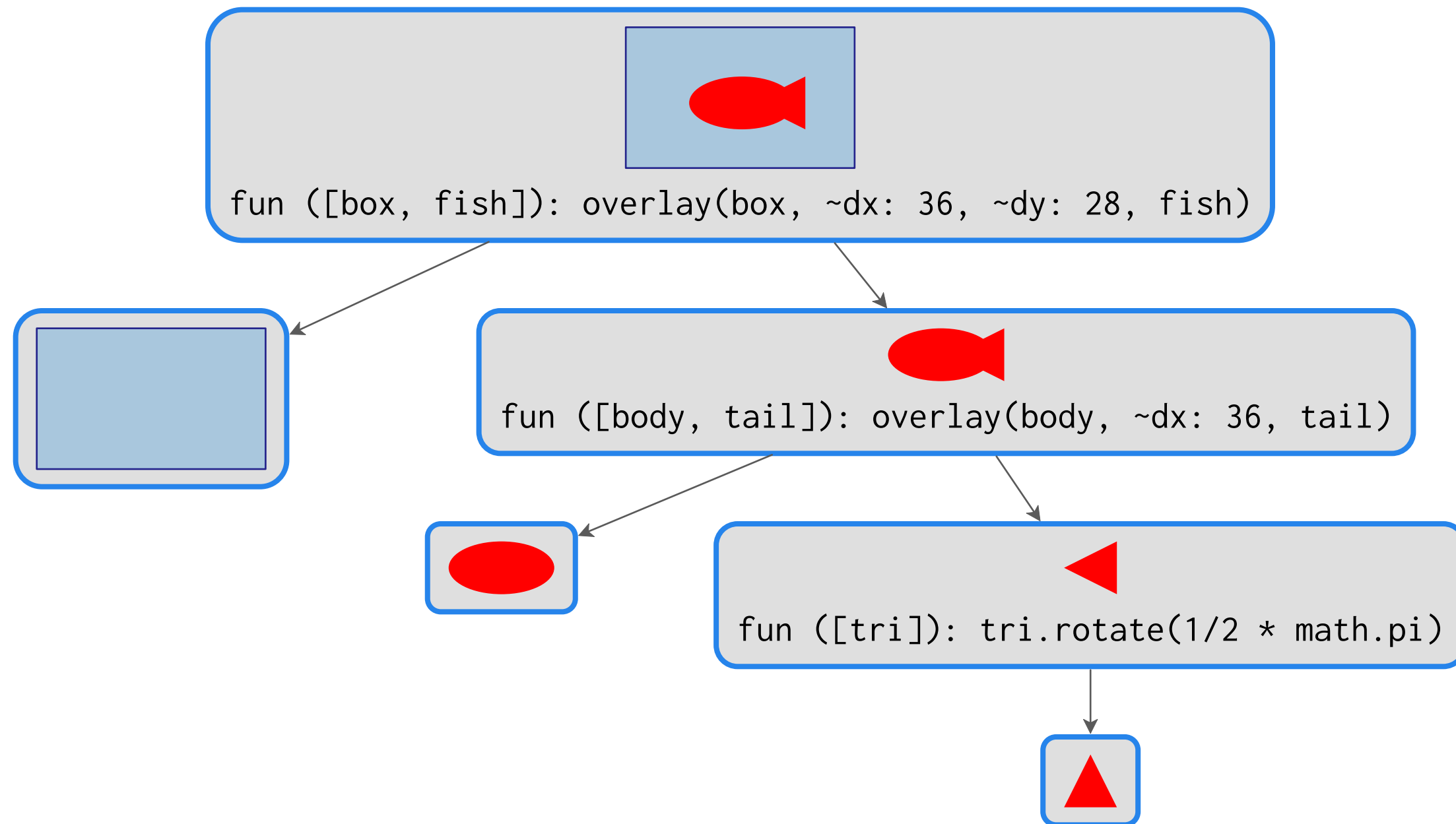
What about Pict?



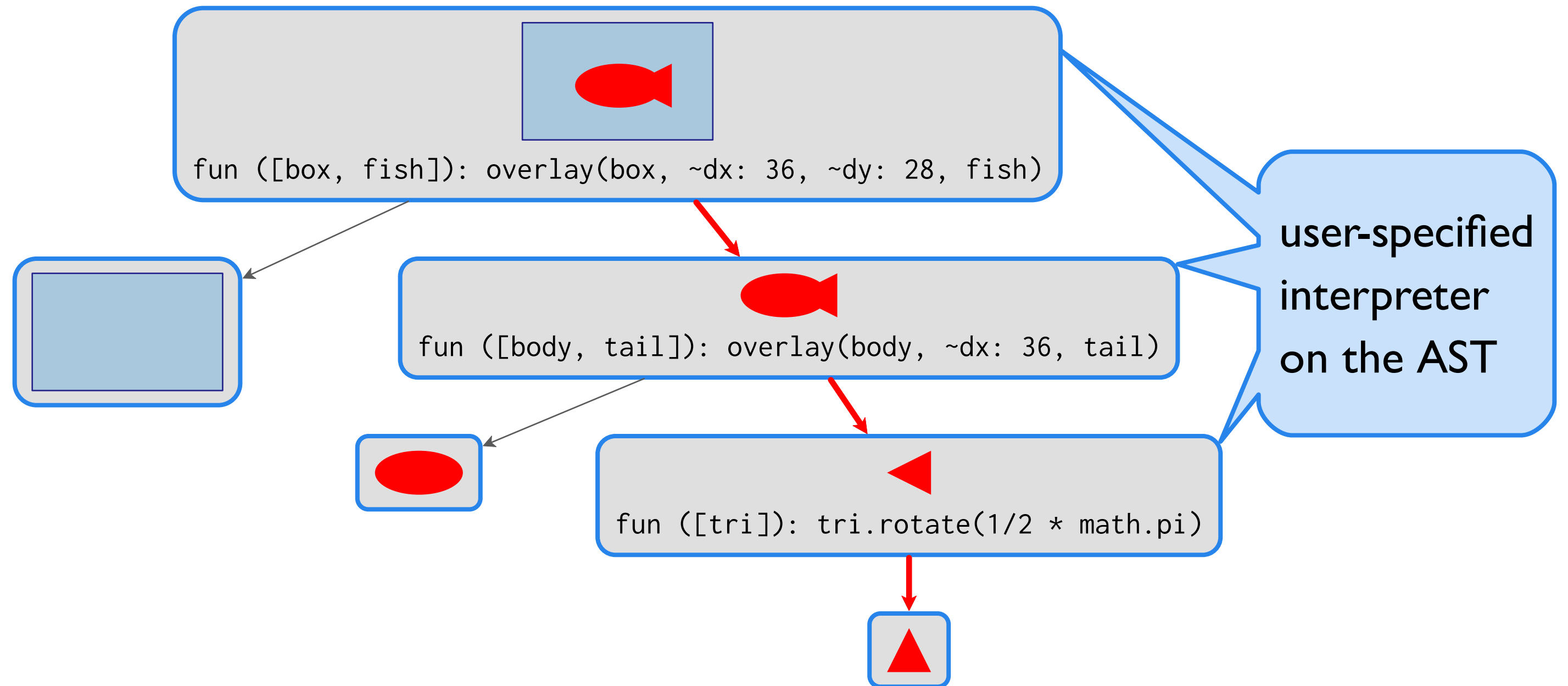
What about Pict?



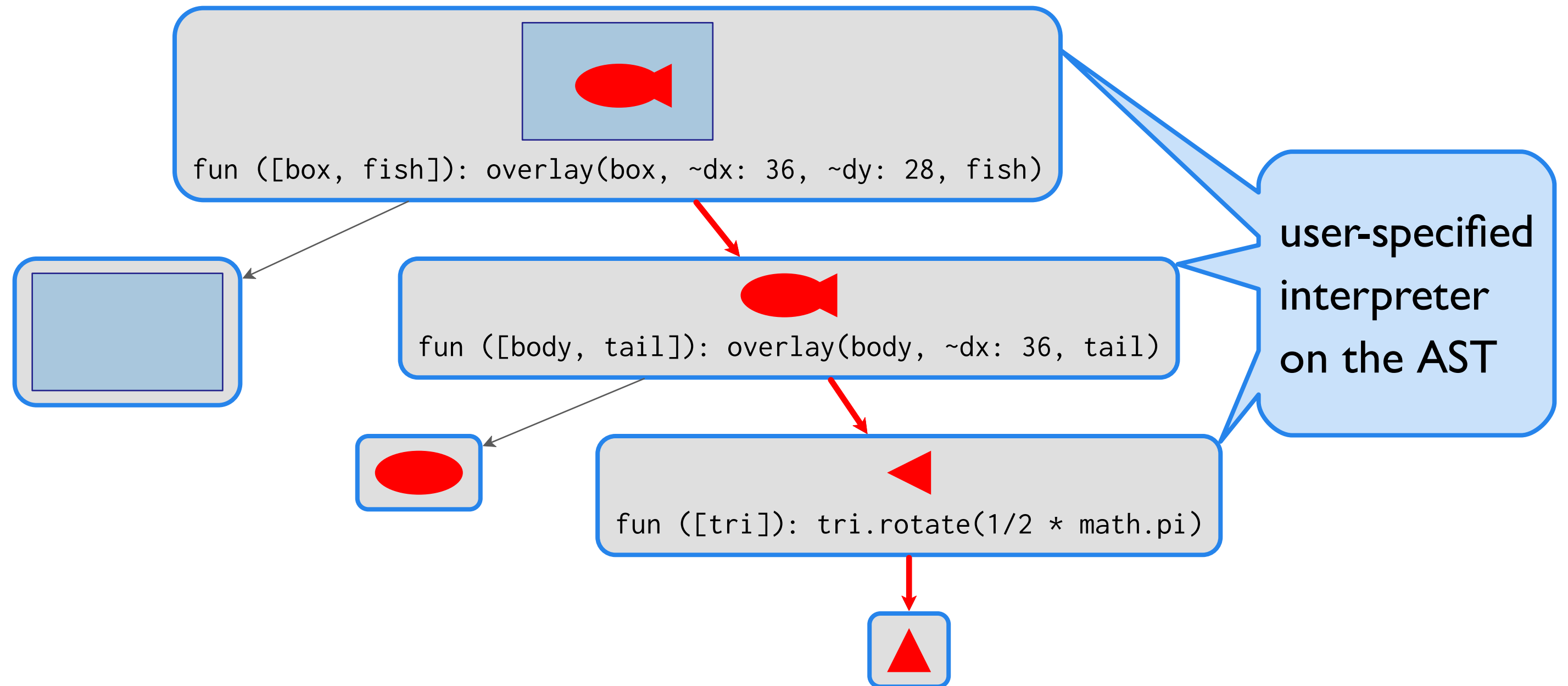
What about Pict?



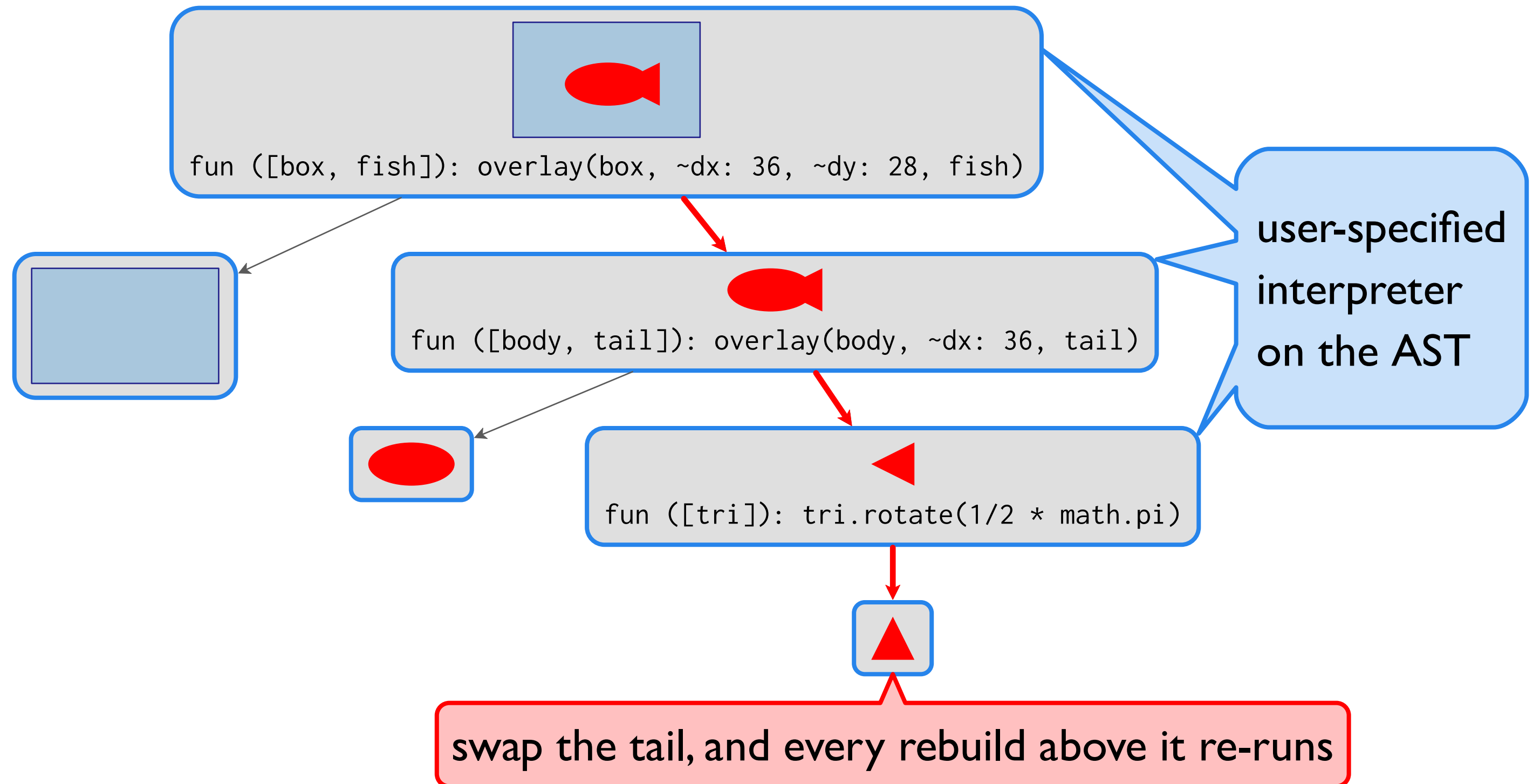
What about Pict?



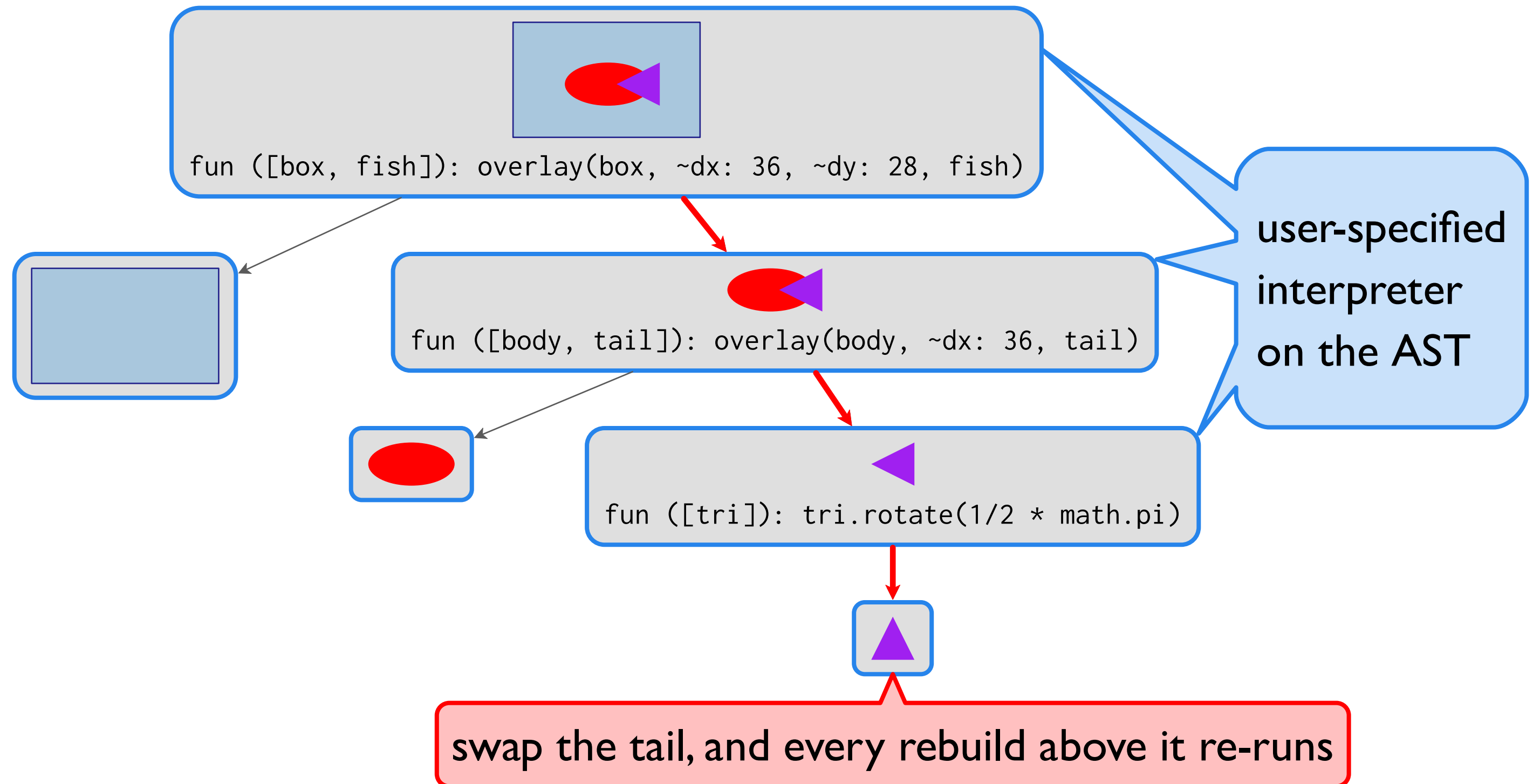
What about Pict?



What about Pict?



What about Pict?

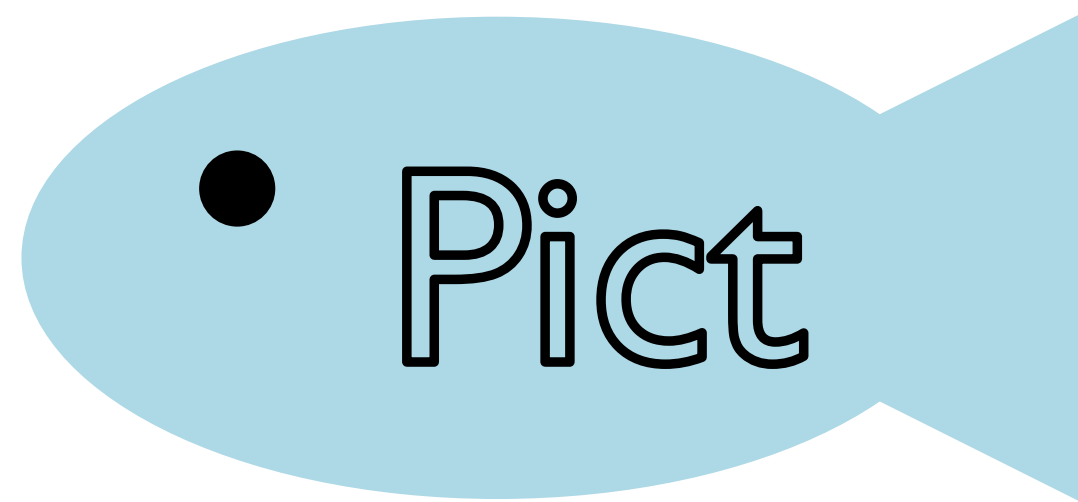


What Next?

What Next?



L^AT_EX + beamer

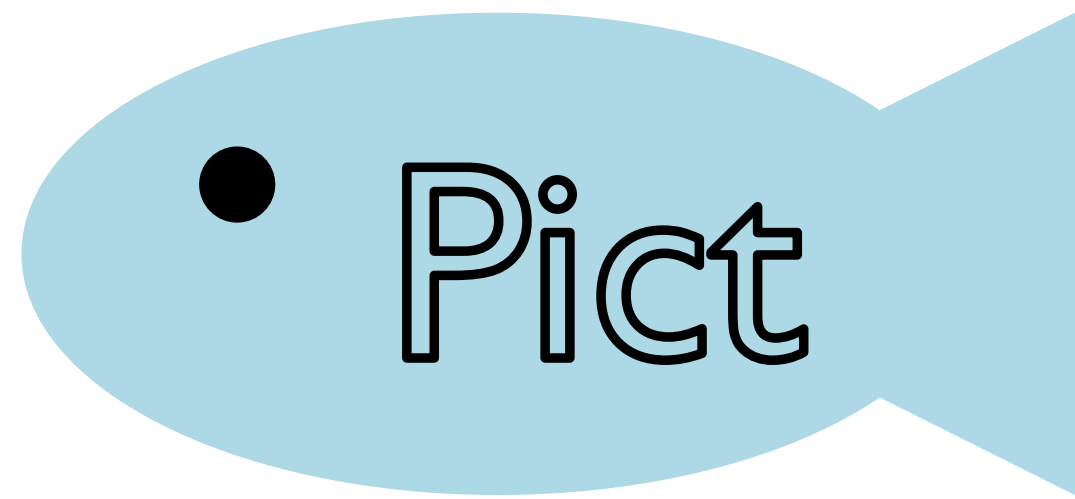


fully composable animations

What Next?



L^AT_EX + beamer



fully composable animations

Animated Pictures for Slide Presentations



Oliver Flatt

University of Washington

Robert Bruce Findler

Northwestern University

Matthew Flatt

University of Utah

Animated Pictures for Slide Presentations



Oliver Flatt

University of Washington

Robert Bruce Findler

Northwestern University

Matthew Flatt

University of Utah

questions?

Animated Pictures for Slide Presentations



Oliver Flatt

University of Washington

Robert Bruce Findler

Northwestern University

Matthew Flatt

University of Utah

Try Flatt today!

<https://rhombus-lang.org/>

questions?

How does pict identity work?

How does pict identity work?

```
def a = square(~size: 80)  
def b = square(~size: 80)  
def p = stack(a, b)
```

